TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

1. Datos informativos

Asignatura:	Teoría de la Programación
Docente:	José Guamán
Estudiante(s):	Iker Alexander Acaro González
Curso / Paralelo:	Ing. En Computación – 1er Ciclo
Fecha de entrega:	03 de noviembre del 2026
Tema:	Desarrollo de algoritmos: representación gráfica, pseudocódigo, prueba de escritorio y codificación en C

2. Objetivo

Desarrollar los algoritmos propuestos por el docente aplicando las distintas formas de representación y construcción de soluciones: diagrama de flujo, pseudocódigo, prueba de escritorio y su implementación en el lenguaje de programación C.

3. Instrucciones generales

1. El trabajo se realizará de forma **individual**
2. Se deben desarrollar **todos los algoritmos enviados** por el docente.
3. Por **cada algoritmo** se debe presentar la estructura indicada en el punto 4, respetando el orden establecido.
4. Los diagramas de flujo pueden elaborarse en herramientas como **draw.io, Flowgorithm o Lucidchart**, o a mano legible (fotografiados).
5. La codificación debe realizarse en **lenguaje C**, compilada y probada. Se debe incluir una **captura de pantalla de la salida** del programa con datos de ejemplo.
6. El documento se entregará en formato **PDF**

7. La copia total o parcial de trabajos será sancionada según el reglamento institucional.
8. El docente podrá solicitar una **sustentación oral** del trabajo para verificar la autoría de las soluciones presentadas.

4. Estructura del trabajo (puntos a desarrollar por cada algoritmo)

- **Enunciado del algoritmo:**

EJ-001 (MEDIO)
En un taller mecanico, declare tres variables enteras llamadas x, u y val2. Solicite por teclado tres numeros enteros (lectura) y almacenelos en esas variables. Calcule la suma, el producto y el promedio (expresion aritmetica) usando una constante literal 3 como divisor. Asigne los resultados a variables suma, producto y promedio. Muestre por pantalla (escritura) un mensaje con los tres resultados. Justifique por que promedio debe ser tipo real mientras que suma y producto son enteros. Ademas del calculo principal, determine si el resultado es positivo o negativo. Agregue un mensaje indicando el tipo de operacion realizada.

- **Análisis del Problema:**

Constante literal: 3.0
Datos de entrada:
Tres variables enteras (x, u y val2)
Proceso:
suma= x + u + val2
producto= x * u * val2
promedio= suma/3.0
Comparar el promedio con 0 usando condicionales para indicar si es positivo o negativo
Datos de salida:
Mensaje con la suma, producto, promedio y signo del promedio
Variables y tipos:
Como enteros x, u, val2, suma y producto
Como real promedio
Constante literal de 3.0

- **Diagrama de flujo:**

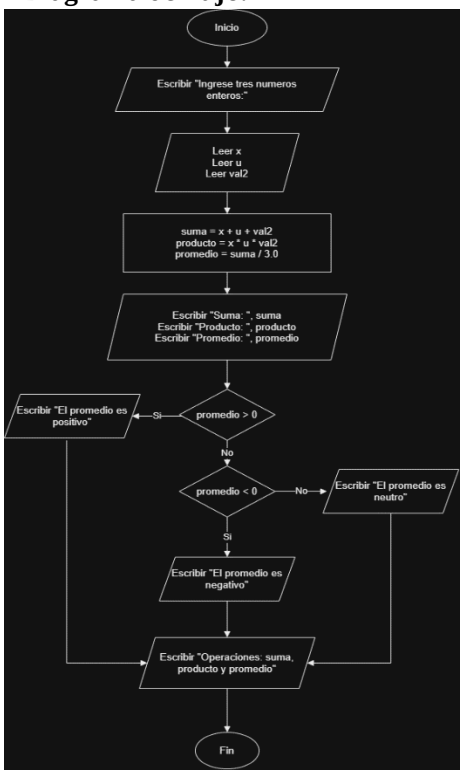


Figura 1. *Diagrama de flujo del EJ-001*
Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-001

- **Pseudocódigo:**

Inicio

```
Definir x, u, val2, suma, producto Como Entero
Definir promedio Como Real
Escribir "Ingrese tres números enteros:"
Leer x
Leer u
Leer val2
suma = x + u + val2
producto = x * u * val2
promedio = suma / 3.0
Escribir "Suma: ", suma
Escribir "Producto: ", producto
Escribir "Promedio: ", promedio
Si promedio > 0 Entonces
   Escribir "El promedio es positivo"
Sino
  Si promedio < 0 Entonces
  Escribir "El promedio es negativo"
  Sino
  Escribir "El promedio es neutro"
  FinSi
FinSi
Fin
```

- **Prueba de Escritorio:**

Casos	x	u	val2	suma	producto	promedio	Condición
1	3	6	0	9	0	3	Positivo (0<3)
2	-1	2	-4	-3	8	-1	Negativo (-1<0)
3	-5	5	0	0	0	0	Neutro (0=0)

Tabla 1. *Prueba de escritorio del EJ-001*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>

void main() {
   int x , u , val2 , suma , producto = 0;
   float promedio = 0.0;
   float DIVISOR = 3.0;
   //Entrada
   printf("Ingrese tres numeros enteros:\n");
   scanf("%d %d %d", &x, &u, &val2);
   //Proceso
   suma = x + u + val2;
   producto = x * u * val2;
   promedio = suma / DIVISOR;
   //Salida
   printf("Suma: %d\nProducto: %d\nPromedio: %.2f\n", suma, producto, promedio);
   if (promedio > 0)
      printf("El promedio es positivo\n");
   else if (promedio < 0)
      printf("El promedio es negativo\n");
   else
      printf("El promedio es cero\n");
}
```

- **Salida del programa**

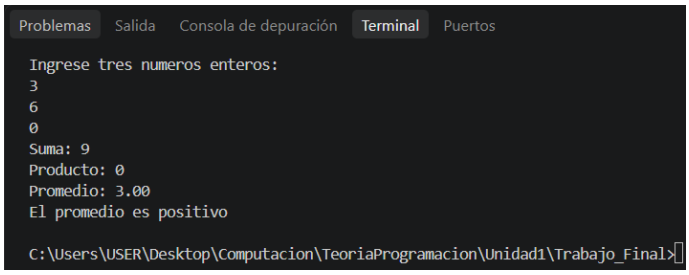


Figura 2. *Salida de la terminal del EJ-001*

Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-001

- **Conclusiones del algoritmo**
  - Utilice las condiciones facilita la tarea de detectar el signo de un numero o el signo de los resultados de un proceso.
  - Comprendí que datos como el promedio es mejor almacenarlos en variables para una mejor organización del código y un buen resultado.

- **Enunciado del algoritmo:**

EJ-002 (MEDIO)

En una libreria, construya un algoritmo que lea una temperatura en grados Celsius (real) por teclado. Use una constante simbolica FACTOR_F = 1.8 y otra OFFSET_F = 32. Calcule la temperatura en Fahrenheit con la expresion aritmetica (celsius * FACTOR_F + OFFSET_F) y asigne el resultado a fahrenheit. Muestre por pantalla un mensaje del tipo 'X grados C equivalen a Y grados F'. Identifique claramente el dato de entrada y la informacion de salida. Ademas del calculo principal, calcule el logaritmo en base 10 del resultado. Muestre al final los valores de todas las variables auxiliares.

- **Análisis del Problema:**

Constantes: FACTOR_F = 1.8 y OFFSET_F = 32

Datos de entrada:

Una temperatura real expresada en grados Celsius (celsius)

Proceso:

fahrenheit = celsius * FACTOR_F + OFFSET_F

Si fahrenheit > 0, logaritmo = log10(fahrenheit)

Datos de salida:

Equivalencia Celsius–Fahrenheit, logaritmo cuando exista y valores de las dos constantes utilizadas.

Variables y tipos:

Como reales celsius, fahrenheit, logaritmo

Como constante real FACTOR_F

Como constante entero OFFSET_F

- **Diagrama de flujo:**

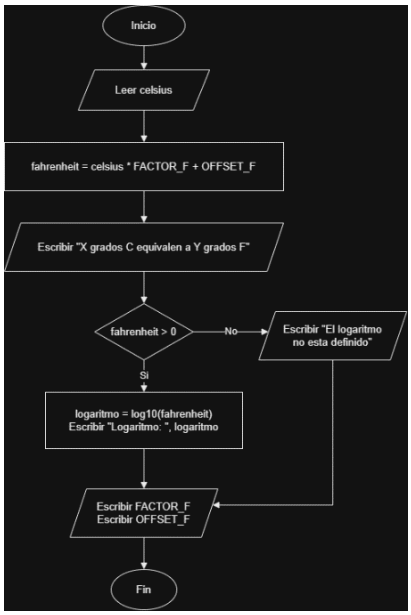


Figura 3. *Diagrama de flujo del EJ-002*

Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-002

- **Pseudocódigo:**

Inicio
Definir celsius, fahrenheit, logaritmo Como Real
Definir FACTOR_F = 1.8 Como Constante Real
Definir OFFSET_F = 32.0
Como Constante Real
Escribir "Ingrese grados Celsius:"
Leer celsius
fahrenheit = celsius * FACTOR_F + OFFSET_F
Escribir celsius, " grados C equivalen a ", fahrenheit, " grados F"
Si fahrenheit > 0 Entonces
logaritmo = log10(fahrenheit)
Escribir "Logaritmo base 10: ", logaritmo
Sino
Escribir "El logaritmo no esta definido"
FinSi
Escribir "FACTOR_F: ", FACTOR_F
Escribir "OFFSET_F: ", OFFSET_F
Fin

- **Prueba de Escritorio:**

Casos	Celsius	Cálculos	Condición	Salida en pantalla
1	0	fahrenheit=32.00 logaritmo≈1.51	fahrenheit > 0 Sí	0.00 °C = 32.00 °F Logaritmo: 1.51
2	-20	fahrenheit=-4.00 No se calcula logaritmo	fahrenheit > 0 No	-20.00 °C = -4.00 °F Logaritmo no definido

Tabla 2. *Prueba de escritorio del EJ-002*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>
#include <math.h>

void main() {
  float FACTOR_F = 1.8;
  float OFFSET_F = 32.0;
  float celsius, fahrenheit, logaritmo = 0.0;

  //Entrada
```

```
    printf("Ingrese los grados Celsius: ");
    scanf("%f", &celsius);
    //Proceso
    fahrenheit = celsius * FACTOR_F + OFFSET_F;
    //Salida
    printf("%.2f grados C equivalen a %.2f grados F\n", celsius, fahrenheit);
    if (fahrenheit > 0) {
        logaritmo = log10(fahrenheit);
        printf("Logaritmo base 10: %.2f\n", logaritmo);
    } else {
        printf("El logaritmo no esta definido para este resultado\n");
    }
    printf("FACTOR_F: %.2f, OFFSET_F: %.2f\n", FACTOR_F, OFFSET_F);
}
```

- **Salida del programa**

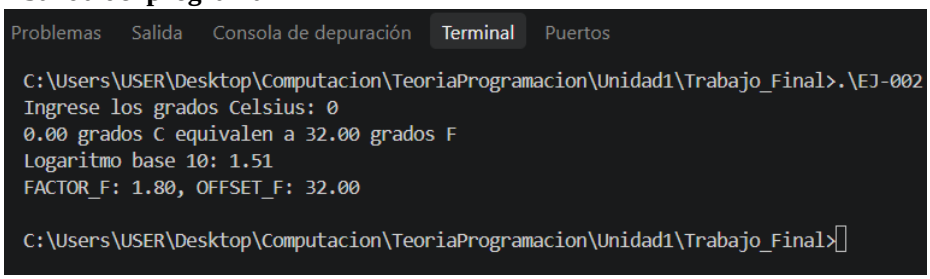


Figura 4. *Salida de la terminal del EJ-002*
Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-002

- **Conclusiones del algoritmo**
  - Utilicé las condicionales para comprobar si es posible sacar el logaritmo evitando el uso de operaciones matemáticas que salgan invalidas.
  - Aprendí a convertir la temperatura de grados Celsius a Fahrenheit usando las fórmulas matemáticas y usando constantes.

- **Enunciado del algoritmo:**

EJ-003 (Medio)
En una empresa de transporte, diseñe un algoritmo que calcule el area de un circulo. Declare una constante simbolica PI = 3.14159 y una variable real radio. Lea el radio por teclado. Calcule el area con la expresion aritmetica (PI * radio * radio) y asignela a la variable area. Muestre por pantalla el resultado con un mensaje descriptivo. Justifique por que se usa constante simbolica y no un literal para PI. Ademas del calculo principal, calcule el resto de una division adicional. Incluya un mensaje de confirmacion al finalizar la operacion.

- **Análisis del Problema:**

Constante simbólica: PI = 3.14159. Se utiliza porque su valor no cambia y se identifica mejor en la fórmula.
Datos de entrada:
Un radio real (radio) y dos números enteros (numero y divisor)
Proceso:
Comprobar que radio >= 0 y divisor != 0
area = PI * radio * radio
residuo = numero MOD divisor
Datos de salida:
Mensaje con el área del círculo, residuo y confirmación de la operación o datos inválidos.
Variables y tipos:
Como reales radio y area
Como enteros numero, divisor y residuo
Como constante real PI

- **Diagrama de flujo:**

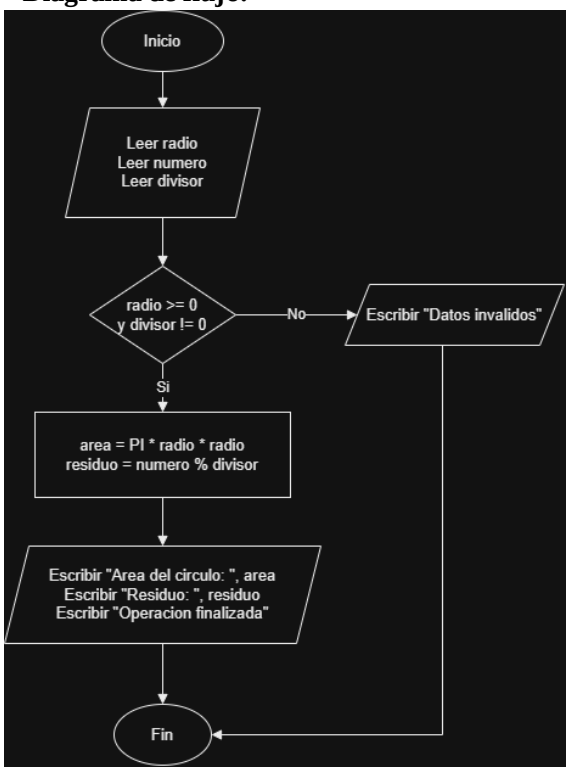


Figura 5. *Diagrama de flujo del EJ-003*

Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-003

- **Pseudocódigo:**

```
Inicio
Definir radio, area Como Real
Definir numero, divisor, residuo Como Entero
Definir PI = 3.14159 Como Constante Real
Leer radio
Escribir "Ingrese un numero entero y un divisor:"
Leer numero, divisor
Si radio >= 0 Y divisor <> 0 Entonces
area = PI * radio * radio
residuo = numero MOD divisor
Escribir "Area del circulo: ", area
Escribir "Residuo de la division: ", residuo
Escribir "Operacion finalizada correctamente"
Sino
Escribir "Datos invalidos"
FinSi
Fin
```

- **Prueba de Escritorio:**

Casos	radio	numero	divisor	area	residuo	Condición
1	2	10	3	12.57	1	Datos válidos. Operación finalizada
2	3	14	4	28.27	2	Datos válidos. Operación finalizada

Tabla 3. *Prueba de escritorio del EJ-003*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>

void main() {
    float PI = 3.14159;
```

```
    float radio, area = 0.0;
    int numero, divisor, residuo = 0;
    //Entrada
    printf("Ingrese el radio: ");
    scanf("%f", &radio);
    printf("Ingrese un numero entero y un divisor: ");
    scanf("%d %d", &numero, &divisor);
    //Proceso y salida
    if (radio >= 0 && divisor != 0) {
        area = PI * radio * radio;
        residuo = numero % divisor;
        printf("Area del circulo: %.2f\nResiduo de la division: %d\nOperacion finalizada correctamente\n", area, residuo);
    } else {
        printf("Datos invalidos: radio negativo o divisor cero\n");
    }
}
```

- **Salida del programa**

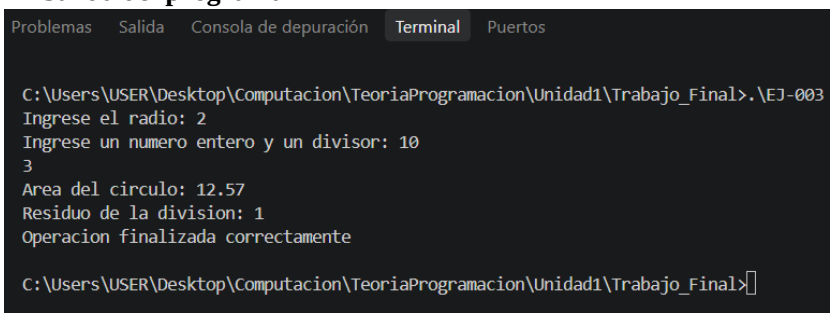


Figura 6. *Salida de la terminal del EJ-003*

Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-003

- **Conclusiones del algoritmo**
  - Comprendí mejor el uso de constantes en valores importantes como lo es el PI
  - Aprendí a sacar el área de un circulo usando su radio y adaptarlo al algoritmo

- **Enunciado del algoritmo:**

EJ-004 (Medio)

En una zapateria, cree un algoritmo que solicite el nombre de una persona (cadena), su año de nacimiento (entero) y el año actual (entero). Use una constante simbolica MAYORIA_EDAD = 18. Calcule la edad con la expresion aritmetica (año_actual - año_nacimiento) y asignela a edad. Evalúe la expresion relacional (edad >= MAYORIA_EDAD) y muestre por pantalla 'X tiene Y años y es mayor/menor de edad'. Ademas, calcule el valor minimo de todas las variables utilizadas. Agregue validaciones para que no haya resultados negativos.

- **Análisis del Problema:**

Constante simbólica: MAYORIA_EDAD = 18

Datos de entrada:

Nombre de una persona y dos años enteros (anio_nacimiento y anio_actual)

Proceso:

Comprobar que los años sean válidos y que no se obtenga una edad negativa.

edad = anio_actual - anio_nacimiento

Comparar edad con MAYORIA_EDAD para indicar si es mayor o menor de edad.

Encontrar el mínimo entre edad, anio_nacimiento y anio_actual.

Datos de salida:

Mensaje con el nombre, edad, mayoría o minoría de edad y valor mínimo; o fechas inválidas.

Variables y tipos:

Como cadena nombre

Como enteros anio_nacimiento, anio_actual, edad y minimo

Como constante entera MAYORIA_EDAD

- **Diagrama de flujo:**

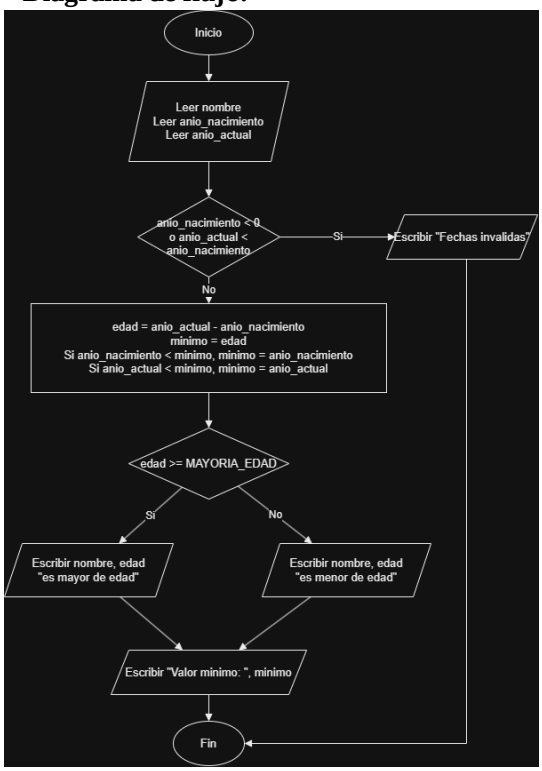


Figura 7. *Diagrama de flujo del EJ-004*
Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-004

- **Pseudocódigo:**

Inicio
Definir nombre Como Cadena
Definir anio_nacimiento, anio_actual, edad, minimo Como Entero
Definir MAYORIA_EDAD = 18 Como Constante Entera
Escribir "Ingrese el nombre:"
Leer nombre
Escribir "Ingrese el año de nacimiento y el año actual:"
Leer anio_nacimiento
Leer anio_actual
Si anio_nacimiento < 0 O anio_actual < anio_nacimiento Entonces
Escribir "Fechas invalidas"
Sino
edad = anio_actual - anio_nacimiento
minimo = edad
Si anio_nacimiento < minimo Entonces
minimo = anio_nacimiento
FinSi
Si anio_actual < minimo Entonces
minimo = anio_actual
FinSi
Si edad >= MAYORIA_EDAD Entonces
Escribir nombre, " tiene ", edad, " años y es mayor de edad"
Sino
Escribir nombre, " tiene ", edad, " años y es menor de edad"
FinSi
Escribir "Valor minimo: ", minimo
FinSi
Fin

- **Prueba de Escritorio:**

Casos	nombre	anio_ nacimiento	anio_ actual	edad	minimo	Condición

Casos	nombre	anio_ nacimiento	anio_ actual	edad	minimo	Condición
1	Maria Lopez	2000	2026	26	26	Mayor de edad (26 >= 18)
2	Luis	2010	2026	16	16	Menor de edad (16 < 18)

Tabla 4. *Prueba de escritorio del EJ-004*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>

void main() {
   int MAYORIA_EDAD = 18;
   char nombre[50] = "";
   int anio_nacimiento, anio_actual, edad, minimo = 0;
   //Entrada
   printf("Ingrese el nombre: ");
   scanf(" %49[^\n]", nombre);
   printf("Ingrese el año de nacimiento y el actual: ");
   scanf("%d %d", &anio_nacimiento, &anio_actual);
   //Proceso y salida
   if (anio_nacimiento < 0 || anio_actual < anio_nacimiento) {
      printf("Fechas invalidas: no se permiten edades negativas\n");
   } else {
      edad = anio_actual - anio_nacimiento;
      minimo = edad;
      if (anio_nacimiento < minimo) minimo = anio_nacimiento;
      if (anio_actual < minimo) minimo = anio_actual;

      if (edad >= MAYORIA_EDAD)
         printf("%s tiene %d años y es mayor de edad\n", nombre, edad);
      else
         printf("%s tiene %d años y es menor de edad\n", nombre, edad);
      printf("Valor minimo entre las variables numericas: %d\n", minimo);
   }
}
```

- **Salida del programa**

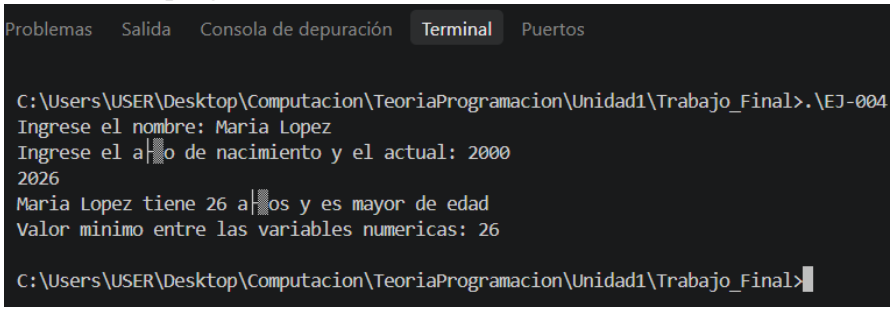


Figura 8. *Salida de la terminal del EJ-004*

Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-004

- **Conclusiones del algoritmo**
  - Comprendí a cómo usar las condicionales para determinar si una persona es mayor o menor de edad según sus datos ingresados.
  - Aprendí a calcular la edad de las personas mediante la resta de la fecha actual con la fecha de nacimiento.

- **Enunciado del algoritmo:**

EJ-005 (Medio)

En una cooperativa, declare una variable entera a y una constante literal entera 7. Lea a por teclado. Calcule el residuo de la division (a % 7) y asigne el resultado a residuo. Evalúe la expresion logica (residuo == 0 AND a > 0). Muestre por pantalla el mensaje 'Es multiplo de 7' o 'No es multiplo de 7'. Indique el tipo de dato simple de cada variable usada. Ademas, determine si el resultado es par o impar (cuando aplique). Incluya mensajes informativos durante la ejecucion del algoritmo.

- **Análisis del Problema:**

Constante literal: 7

Datos de entrada:

Un número entero (a)

Proceso:

residuo = a MOD 7

Comprobar si residuo == 0 y a > 0

Comprobar residuo MOD 2 para indicar si el residuo es par o impar.

Datos de salida:

Mensaje con el residuo, si es múltiplo de 7 y si el residuo es par o impar.

Variables y tipos:

Como enteros a y residuo

Constante literal entera de 7

- **Diagrama de flujo:**

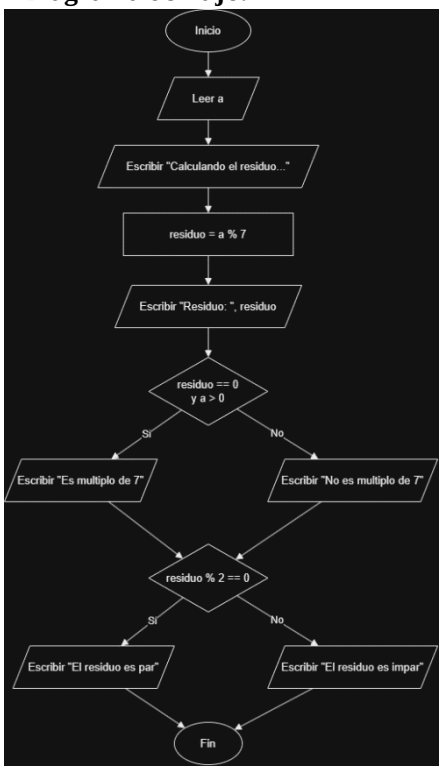


Figura 9. *Diagrama de flujo del EJ-005*

Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-005

- **Pseudocódigo:**

Inicio
Definir a, residuo Como Entero
Escribir "Ingrese un numero entero:"
Leer a
Escribir "Calculando el residuo y verificando el resultado"
residuo = a MOD 7
Escribir "Residuo: ", residuo
Si residuo == 0 Y a > 0 Entonces
Escribir "Es multiplo de 7"
Sino
Escribir "No es múltiplo de 7"

FinSi
Si residuo MOD 2 == 0 Entonces
Escribir "El residuo es par"
Sino
Escribir "El residuo es impar"
FinSi
Fin

- **Prueba de Escritorio:**

Casos	a	residuo	residuo % 2	Condición
1	21	0	0	Múltiplo de 7 y residuo par
2	10	3	1	No múltiplo de 7 y residuo impar

Tabla 5. *Prueba de escritorio del EJ-005*
Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>

void main() {
   int a, residuo = 0;
   const int DIVISOR = 7;
   //Entrada
   printf("Ingrese un numero entero: ");
   scanf("%d", &a);
   //Proceso
   residuo = a % DIVISOR;
   //Salida
   printf("Residuo: %d\n", residuo);
   if (residuo == 0 && a > 0)
      printf("Es multiplo de 7\n");
   else
      printf("No es multiplo de 7\n");
   if (residuo % 2 == 0)
      printf("El residuo es par\n");
   else
      printf("El residuo es impar\n");
}
```

- **Salida del programa**

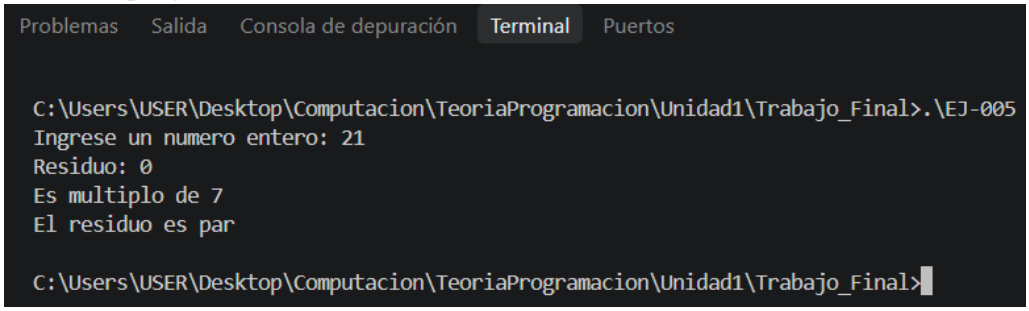


Figura 10. *Salida de la terminal del EJ-005*
Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-005

- **Conclusiones del algoritmo**
  - Aprendí a como usar el operador % para lograr obtener el residuo de una división y comprobar si un numero es múltiplo de 7.
  - Comprendí como funciona el operador && que permite comprobar si dos condiciones se cumplen al mismo tiempo.

- **Enunciado del algoritmo:**

EJ-051 (Avanzado)

En una joyeria, diseñe un algoritmo que valide una contraseña ingresada por teclado (cadena). Use constantes simbolicas MIN_LONGITUD = 8, MAX_LONGITUD = 20, REQUIERE_MAYUS = 1, REQUIERE_DIGITO = 1. Evalúe la expresion logica compuesta: (longitud(contraseña) >= MIN_LONGITUD AND longitud(contraseña) <= MAX_LONGITUD AND contiene_mayuscula(contraseña) == REQUIERE_MAYUS AND contiene_digito(contraseña) == REQUIERE_DIGITO). Asigne el resultado a una variable booleana es_valida. Muestre por pantalla mensajes indicando que criterios se cumplieron. Identifique cada tipo de dato simple utilizado. Ademas, calcule el valor minimo de todas las variables utilizadas. Identifique explicitamente los datos de entrada y la informacion de salida.

- **Análisis del Problema:**

Constantes simbólicas: MIN_LONGITUD = 8, MAX_LONGITUD = 20, REQUIERE_MAYUS = 1 y REQUIERE_DIGITO = 1

Datos de entrada:

Una contraseña (cadena de caracteres)

Proceso:

Obtener la longitud de la contraseña

Comprobar si tiene una mayúscula y un dígito

es_valida = (longitud >= 8 Y longitud <= 20 Y tiene_mayus == 1 Y tiene_digito == 1)

Encontrar el mínimo entre los valores numéricos utilizados

Datos de salida:

Mensajes con los criterios cumplidos, si la contraseña es válida y el mínimo numérico.

Variables y tipos:

Como cadena contrasena

Como enteros i, longitud, tiene_mayus, tiene_digito y minimo

Como lógico es_valida

Como constantes enteras MIN_LONGITUD, MAX_LONGITUD, REQUIERE_MAYUS y REQUIERE_DIGITO

- **Diagrama de flujo:**

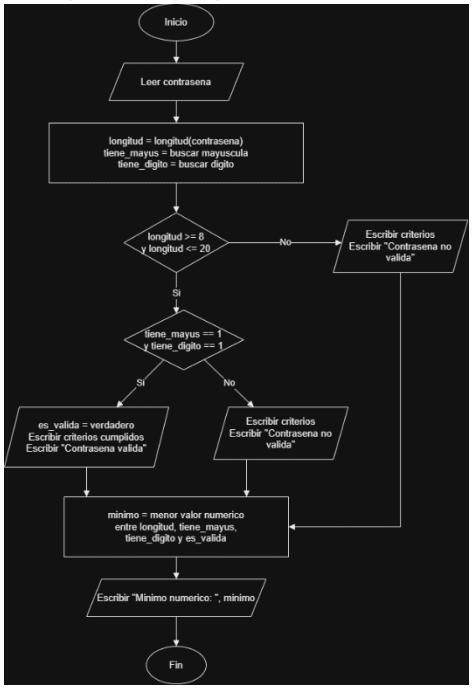


Figura 11. *Diagrama de flujo del EJ-0051*

Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-051

- **Pseudocódigo:**

Inicio

Definir contrasena Como Cadena

Definir longitud, tiene_mayus, tiene_digito, minimo Como Entero

Definir es_valida Como Logico

Definir MIN_LONGITUD = 8, MAX_LONGITUD = 20 Como Constantes Enteras

Definir REQUIERE_MAYUS = 1, REQUIERE_DIGITO = 1 Como Constantes Enteras

```
longitud = 0
tiene_mayus = 0
tiene_digito = 0
Escribir "Ingrese la contrasena:"
Leer contrasena
longitud = Longitud(contrasena)
Si ContieneMayuscula(contrasena) Entonces
tiene_mayus = 1
FinSi
Si ContieneDigito(contrasena) Entonces
tiene_digito = 1
FinSi
es_valida = (longitud >= MIN_LONGITUD Y longitud <= MAX_LONGITUD Y
tiene_mayus == REQUIERE_MAYUS Y tiene_digito == REQUIERE_DIGITO)
minimo = longitud
Si tiene_mayus < minimo Entonces
minimo = tiene_mayus
FinSi
Si tiene_digito < minimo Entonces
minimo = tiene_digito
FinSi
Si es_valida < minimo Entonces
   minimo = es_valida
FinSi
Si longitud >= MIN_LONGITUD Entonces
Escribir "Longitud minima: Cumple"
Sino
Escribir "Longitud minima: No cumple"
FinSi
Si longitud <= MAX_LONGITUD Entonces
Escribir "Longitud maxima: Cumple"
Sino
Escribir "Longitud maxima: No cumple"
FinSi
Si tiene_mayus == 1 Entonces
Escribir "Mayuscula: Cumple"
Sino Escribir "Mayuscula: No cumple" FinSi
Si tiene_digito == 1 Entonces
Escribir "Digito: Cumple"
Sino Escribir "Digito: No cumple"
FinSi
Si es_valida Entonces
Escribir "Contrasena: Valida"
Sino Escribir "Contrasena: No valida"
FinSi
Escribir "Minimo numerico: ", minimo
Fin
```

- **Prueba de Escritorio:**

Casos	contraseña	longitud	tiene_mayus	tiene_digito	es_valida	minimo	Condición
1	Clave123A	9	1	1	Sí	1	Cumple todos los criterios
2	clave123	8	0	1	No	0	No cumple mayúscula

Tabla 6. *Prueba de escritorio del EJ-051*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

void main() {
   const int MIN_LONGITUD = 8;
   const int MAX_LONGITUD = 20;
   const int REQUIERE_MAYUS = 1;
   const int REQUIERE_DIGITO = 1; const int DIVISOR = 2.0;
   char contrasena[101] = "";
   int longitud = 0, tiene_mayus = 0, tiene_digito = 0, minimo = 0;
bool es_valida = false;

   // Entrada
   printf("Ingrese la contrasena: ");
   scanf("%100s", contrasena);
   // Proceso
   longitud = strlen(contrasena);
   if (strpbrk(contrasena, "ABCDEFGHIJKLMNOPQRSTUVWXYZ") != NULL)
      tiene_mayus = 1;
   if (strpbrk(contrasena, "0123456789") != NULL)
      tiene_digito = 1;
   if (longitud >= MIN_LONGITUD && longitud <= MAX_LONGITUD &&
      tiene_mayus == REQUIERE_MAYUS && tiene_digito == REQUIERE_DIGITO)
      es_valida = true;
   minimo = longitud;
   if (tiene_mayus < minimo) minimo = tiene_mayus;
   if (tiene_digito < minimo) minimo = tiene_digito;
   if (es_valida < minimo) minimo = es_valida;

   // Salida
   if (longitud >= MIN_LONGITUD)
      printf("Longitud minima: Cumple\n");
   else
      printf("Longitud minima: No cumple\n");
   if (longitud <= MAX_LONGITUD)
      printf("Longitud maxima: Cumple\n");
   else
      printf("Longitud maxima: No cumple\n");
   if (tiene_mayus == 1)
      printf("Mayuscula: Cumple\n");
   else
      printf("Mayuscula: No cumple\n");
   if (tiene_digito == 1)
      printf("Digito: Cumple\n");
   else
      printf("Digito: No cumple\n");
   if (es_valida)
      printf("Contrasena: Valida\n");
   else
      printf("Contrasena: No valida\n");
   printf("Minimo numerico: %d\n", minimo);
}
```

- **Salida del programa**

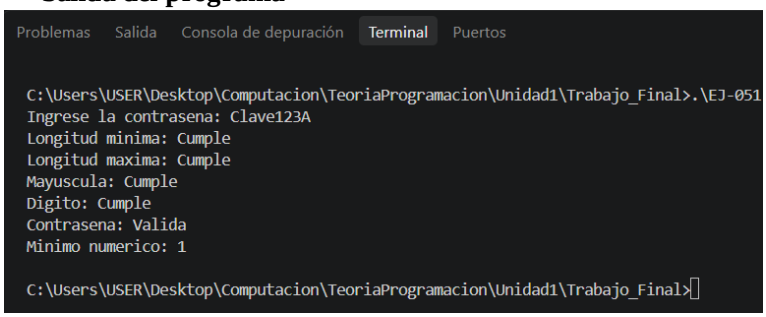


Figura 12. *Salida de la terminal del EJ-051*
Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-051

- **Conclusiones del algoritmo**
  - Aprendí a usar cadenas de caracteres y a como comprobar si una contraseña cumple con la longitud mínima o máxima.
  - Comprendí como verificar diferentes requisitos como el uso de letras mayúsculas y números para determinar si la contraseña es válida.

- **Enunciado del algoritmo:**

EJ-052 (Avanzado)
En una empresa de transporte, construya un algoritmo que simule una calculadora. Lea dos numeros reales n y r, y un operador aritmetico (caracter): '+', '-', '*', '/'. Use expresiones relacionales para seleccionar la operacion. Valide division por cero con la expresion logica (operador == '/' AND r == 0.0). Asigne el resultado a una variable real resultado. Use constantes literales para los operadores. Muestre por pantalla la operacion completa y el resultado. Ademas, calcule el valor minimo de todas las variables utilizadas. Asegurese de inicializar todas las variables antes de usarlas.

- **Análisis del Problema:**

Constantes literales: +, -, * y /
Datos de entrada:
Dos números reales (n y r) y un operador aritmético (operador)
Proceso:
Comprobar que el operador sea válido y evitar la división entre cero
Según el operador, calcular resultado = n + r, n - r, n * r o n / r
Encontrar el mínimo entre n, r y resultado
Datos de salida:
Mensaje con la operación completa, resultado y mínimo o aviso de operador inválido o división entre cero
Variables y tipos:
Como reales n, r, resultado y mínimo
Como carácter operador
Constantes literales de carácter +, -, * y /

- **Diagrama de flujo:**

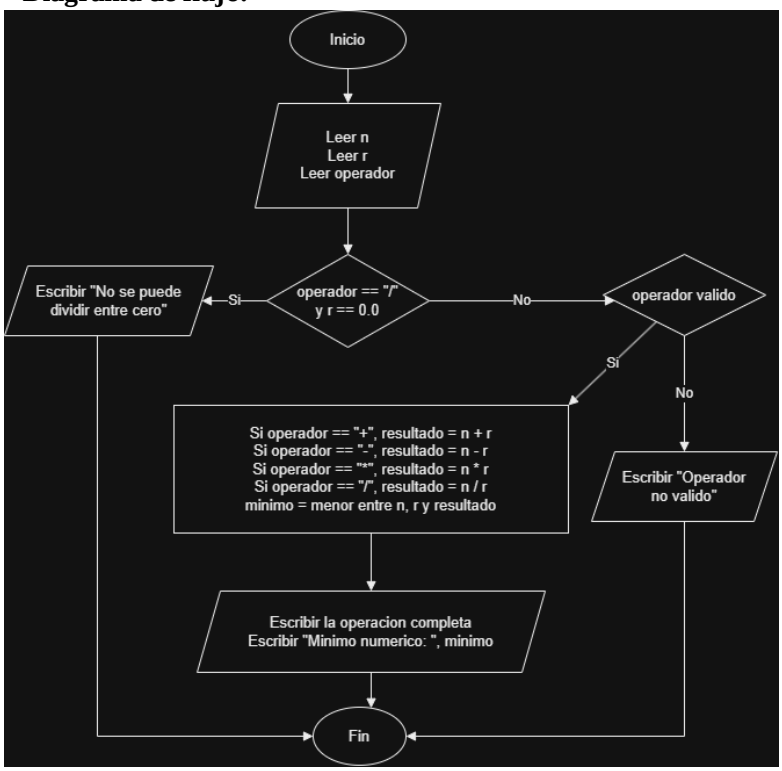


Figura 13. *Diagrama de flujo del EJ-052*
Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-052

- **Pseudocódigo:**

Inicio
Definir n, r, resultado, minimo Como Real
Definir operador Como Caracter
Escribir "Ingrese n, r y operador:"
Leer n
Leer r
Leer operador
Si (operador == '/') Y (r == 0) Entonces
Escribir "No se puede dividir entre cero"
Sino
Si operador no es '+', '-', '*' ni '/' Entonces
Escribir "Operador no valido"
Sino
Si operador == '+' Entonces resultado = n + r
Sino Si operador == '-' Entonces resultado = n - r
Sino Si operador == '*' Entonces resultado = n * r
Sino resultado = n / r
FinSi
minimo = n
Si r < minimo Entonces minimo = r FinSi
Si resultado < minimo Entonces minimo = resultado FinSi
Escribir n, operador, r, " = ", resultado
Escribir "Minimo numerico: ", minimo
FinSi
FinSi
Fin

- **Prueba de Escritorio:**

Casos	n	r	operador	resultado	minimo	Condición

Casos	n	r	operador	resultado	minimo	Condición
1	10	4	+	14.00	4.00	Operador válido. Sin división por cero
2	12	3	/	4.00	3.00	Operador válido (r != 0)

Tabla 7. *Prueba de escritorio del EJ-052*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>

void main() {
    float n = 0, r = 0, resultado = 0, minimo = 0;
    const char SUMA = '+';
    const char RESTA = '-';
    const char MULTIPLICACION = '*';
    const char DIVISION = '/';
    char operador = 0;
    //Entrada
    printf("Ingrese n, r y el operador (+, -, *, /): ");
    scanf("%f %f %c", &n, &r, &operador);
    //Proceso y salida
    if (operador == DIVISION && r == 0.0) {
        printf("No se puede dividir entre cero\n");
    } else if (operador != SUMA && operador != RESTA &&
            operador != MULTIPLICACION && operador != DIVISION) {
        printf("Operador no valido\n");
    } else {
        if (operador == SUMA) resultado = n + r;
        else if (operador == RESTA) resultado = n - r;
        else if (operador == MULTIPLICACION) resultado = n * r;
        else resultado = n / r;
        minimo = n;
        if (r < minimo) minimo = r;
        if (resultado < minimo) minimo = resultado;
        printf("Operacion: %.2f %c %.2f = %.2f\nMinimo numerico: %.2f\n",
            n, operador, r, resultado, minimo);
    }
}
```

- **Salida del programa**

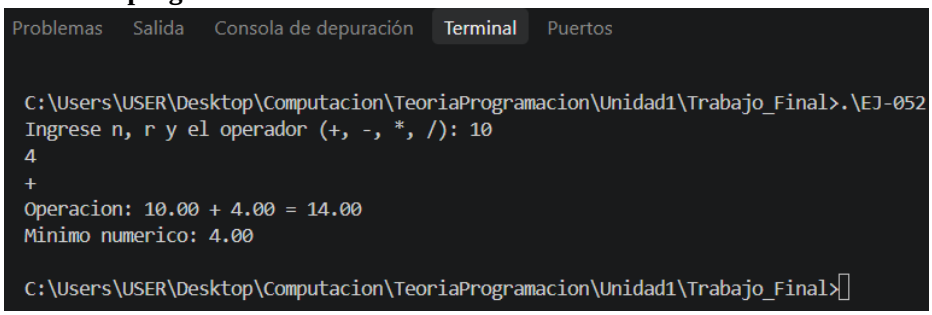


Figura 14. *Salida de la terminal del EJ-052*

Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-052

- **Conclusiones del algoritmo**
  - Aprendí a crear calculadoras sencillas usando condiciones para la selección de operaciones matemáticas.
  - Comprendí la importancia de validad los datos antes de realizar la división, ya que dividir entre cero produciría un error.

- **Enunciado del algoritmo:**

EJ-053 (Avanzado)

En un cine, diseñe un algoritmo que lea una nota (real entre 0 y 100) y la convierta a letra segun escala: A (90-100), B (80-89), C (70-79), D (60-69), F (0-59). Use constantes simbolicas para los limites. Evalúe las expresiones logicas relacionales para asignar la letra correspondiente a una variable de tipo caracter llamada calificacion. Asigne tambien el mensaje descriptivo a una variable cadena. Muestre por pantalla la nota y la letra. Ademas, calcule el valor minimo de todas las variables utilizadas. Incluya una variable auxiliar para llevar un contador de operaciones.

- **Análisis del Problema:**

Constantes simbólicas: LIMITE_A = 90, LIMITE_B = 80, LIMITE_C = 70, LIMITE_D = 60, MIN_NOTA = 0 y MAX_NOTA = 100.

Datos de entrada:

Una nota real entre 0 y 100.

Proceso:

Comprobar que la nota esté entre 0 y 100

Comparar la nota con los límites para asignar A, B, C, D o F.

Guardar el mensaje descriptivo, aumentar el contador y encontrar el mínimo entre nota y contador

Datos de salida:

Mensaje con la nota, letra, descripción, mínimo numérico y contador de operaciones, o nota inválida.

Variables y tipos:

Como reales nota y minimo

Como entero contador

Como carácter calificacion

Como cadena mensaje

Como constantes reales LIMITE_A, LIMITE_B, LIMITE_C, LIMITE_D, MIN_NOTA y MAX_NOTA

- **Diagrama de flujo:**

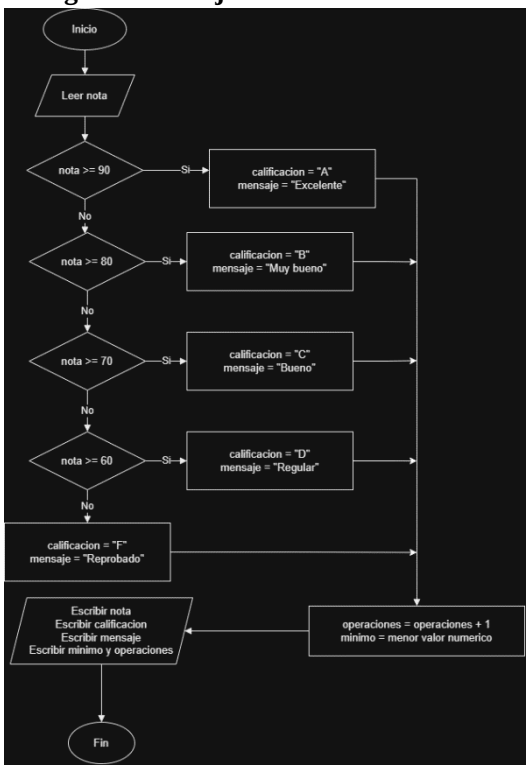


Figura 15. *Diagrama de flujo del EJ-053*

Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-053

- **Pseudocódigo:**

Inicio

Definir nota, minimo Como Real

Definir contador Como Entero

Definir calificacion Como Caracter

Definir mensaje Como Cadena

Definir LIMITE_A=90, LIMITE_B=80, LIMITE_C=70, LIMITE_D=60 Como Constantes

```
Definir MIN_NOTA=0, MAX_NOTA=100 Como Constantes
contador = 0
Escribir "Ingrese la nota de 0 a 100:"
Leer nota
contador = contador + 1
Si nota < MIN_NOTA O nota > MAX_NOTA Entonces
Escribir "Nota fuera del rango 0 a 100"
Sino
Si nota >= LIMITE_A Entonces calificacion='A'; mensaje="Excelente"
Sino Si nota >= LIMITE_B Entonces calificacion='B'; mensaje="Muy bueno"
Sino Si nota >= LIMITE_C Entonces calificacion='C'; mensaje="Bueno"
Sino Si nota >= LIMITE_D Entonces calificacion='D'; mensaje="Regular"
Sino calificacion='F'; mensaje="Insuficiente"
FinSi
contador = contador + 1
minimo = nota
Si contador < minimo Entonces minimo = contador FinSi
Escribir nota, calificacion, mensaje
Escribir minimo, contador
FinSi
Fin
```

- **Prueba de Escritorio:**

Casos	nota	calificacion	mensaje	contador	minimo	Condición
1	95	A	Excelente	2	2.00	95 >= 90
2	0.5	F	Insuficiente	2	0.50	0.5 < 60

Tabla 8. *Prueba de escritorio del EJ-053*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>

void main() {
   const float LIMITE_A = 90;
   const float LIMITE_B = 80;
   const float LIMITE_C = 70;
   const float LIMITE_D = 60;
   const float MIN_NOTA = 0;
   const float MAX_NOTA = 100;
   float nota, minimo = 0.0;
   int contador = 0;
   char calificacion = ' ';
   const char *mensaje = "";
   //Entrada
   printf("Ingrese la nota (0 a 100): ");
   scanf("%f", &nota);
   //Proceso y salida
   contador++;
   if (nota < MIN_NOTA || nota > MAX_NOTA) {
      printf("Nota fuera del rango 0 a 100\n");
   } else {
      if (nota >= LIMITE_A) { calificacion = 'A'; mensaje = "Excelente"; }
      else if (nota >= LIMITE_B) { calificacion = 'B'; mensaje = "Muy bueno"; }
      else if (nota >= LIMITE_C) { calificacion = 'C'; mensaje = "Bueno"; }
      else if (nota >= LIMITE_D) { calificacion = 'D'; mensaje = "Regular"; }
      else { calificacion = 'F'; mensaje = "Insuficiente"; }
```

```
        contador++;
        minimo = nota;
        if (contador < minimo) minimo = contador;
        printf("Nota: %.2f\nCalificacion: %c\nMensaje: %s\n", nota, calificacion, mensaje);
        printf("Minimo numerico: %.2f\nOperaciones: %d\n", minimo, contador);
    }
}
```

- **Salida del programa**

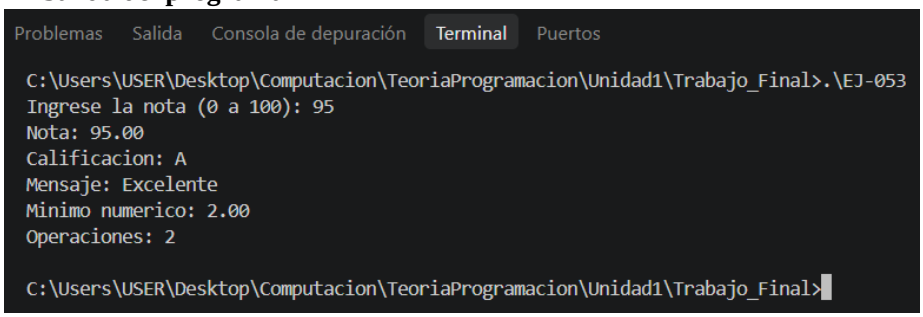


Figura 16. *Salida de la terminal del EJ-053*

Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-053

- **Conclusiones del algoritmo**
  - Aprendí a usar las condicionales para clasificar las calificaciones por rangos.
  - Comprendí que el orden de las condiciones es importante, porque una comparación que este mal ubicada puede provocar una mala asignación.

- **Enunciado del algoritmo:**

EJ-054 (Avanzado)

En una zapateria, cree un algoritmo que lea el peso (real, kg) y la altura (real, m). Use constantes simbolicas BAJO_PESO = 18.5, NORMAL_MAX = 24.9, SOBREPESO_MAX = 29.9, OBESIDAD_MAX = 34.9, OBESIDAD_SEVERA = 39.9. Calcule el IMC con (peso / (altura * altura)). Use expresiones logicas relacionales para clasificar en una de 6 categorias. Asigne la categoria como cadena a una variable. Muestre por pantalla el IMC y la categoria. Ademas, calcule el valor minimo de todas las variables utilizadas. Agregue al final un resumen de los datos ingresados.

- **Análisis del Problema:**

Constantes simbólicas: BAJO_PESO = 18.5, NORMAL_MAX = 24.9, SOBREPESO_MAX = 29.9, OBESIDAD_MAX = 34.9 y OBESIDAD_SEVERA = 39.9

Datos de entrada:

Dos números reales: peso en kg y altura en m

Proceso:

imc = peso / (altura * altura)

Comparar el IMC con los límites para indicar su categoría

Encontrar el mínimo entre peso, altura e imc

Datos de salida:

Mensaje con el IMC, categoría, mínimo numérico y resumen de los datos o aviso si los datos no son positivos.

Variables y tipos:

Como reales peso, altura, imc y minimo

Como cadena categoria

Como constantes reales BAJO_PESO, NORMAL_MAX, SOBREPESO_MAX, OBESIDAD_MAX y OBESIDAD_SEVERA

- **Diagrama de flujo:**

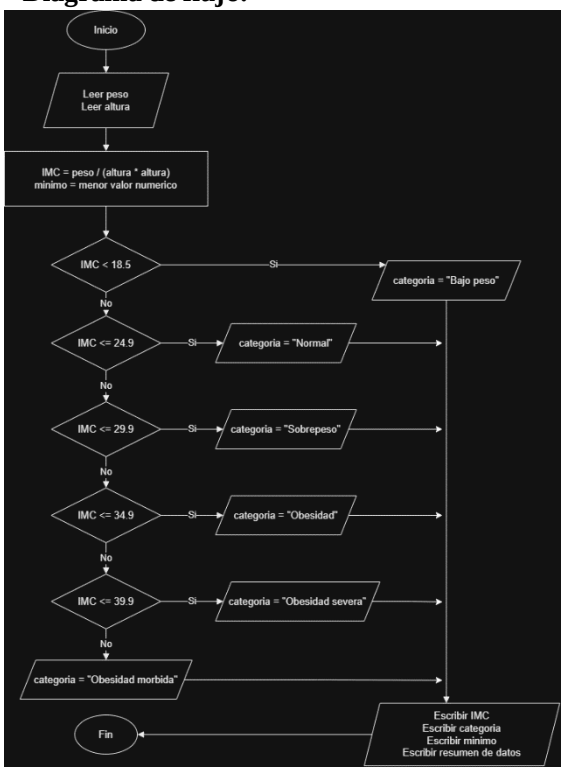


Figura 17. *Diagrama de flujo del EJ-054*
Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-054

- **Pseudocódigo:**

Inicio
Definir peso, altura, imc, minimo Como Real
Definir categoria Como Cadena
Definir BAJO_PESO=18.5, NORMAL_MAX=24.9 Como Constantes
Definir SOBREPESO_MAX=29.9, OBESIDAD_MAX=34.9 Como Constantes
Definir OBESIDAD_SEVERA=39.9 Como Constante
Escribir "Ingrese peso y altura:"
Leer peso
Leer altura
Si peso > 0 Y altura > 0 Entonces
imc = peso / (altura * altura)
Si imc < BAJO_PESO Entonces categoria = "Bajo peso"
Sino Si imc <= NORMAL_MAX Entonces categoria = "Normal"
Sino Si imc <= SOBREPESO_MAX Entonces categoria = "Sobrepeso"
Sino Si imc <= OBESIDAD_MAX Entonces categoria = "Obesidad"
Sino Si imc <= OBESIDAD_SEVERA Entonces categoria = "Obesidad severa"
Sino categoria = "Obesidad extrema"
FinSi
minimo = peso
Si altura < minimo Entonces minimo = altura FinSi
Si imc < minimo Entonces minimo = imc FinSi
Escribir "IMC: ", imc, " Categoria: ", categoria
Escribir "Minimo numerico: ", minimo
Sino
Escribir "Peso y altura deben ser positivos"
FinSi
Escribir "Resumen de datos: ", peso, altura
Fin

- **Prueba de Escritorio:**

Casos	peso	altura	imc	categoria	minimo	Condición

Casos	peso	altura	imc	categoria	minimo	Condición
1	70	1.75	22.86	Normal	1.75	18.5 <= IMC <= 24.9
2	95	1.75	31.02	Obesidad	1.75	29.9 < IMC <= 34.9

Tabla 9. *Prueba de escritorio del EJ-0054*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>

void main() {
    const float BAJO_PESO = 18.5;
    const float NORMAL_MAX = 24.9;
    const float SOBREPESO_MAX = 29.9;
    const float OBESIDAD_MAX = 34.9;
    const float OBESIDAD_SEVERA = 39.9;
    float peso, altura, imc, minimo = 0.0;
    const char *categoria = "";
    //Entrada
    printf("Ingrese peso (kg) y altura (m): ");
    scanf("%f %f", &peso, &altura);
    //Proceso y salida
    if (peso > 0 && altura > 0) {
        imc = peso / (altura * altura);
        if (imc < BAJO_PESO) categoria = "Bajo peso";
        else if (imc <= NORMAL_MAX) categoria = "Normal";
        else if (imc <= SOBREPESO_MAX) categoria = "Sobrepeso";
        else if (imc <= OBESIDAD_MAX) categoria = "Obesidad";
        else if (imc <= OBESIDAD_SEVERA) categoria = "Obesidad severa";
        else categoria = "Obesidad extrema";
        minimo = peso;
        if (altura < minimo) minimo = altura;
        if (imc < minimo) minimo = imc;
        printf("IMC: %.2f\nCategoria: %s\nMinimo numerico: %.2f\n", imc, categoria, minimo);
    } else {
        printf("Peso y altura deben ser positivos\n");
    }
    printf("Resumen: peso %.2f kg, altura %.2f m\n", peso, altura);
}
```

- **Salida del programa**

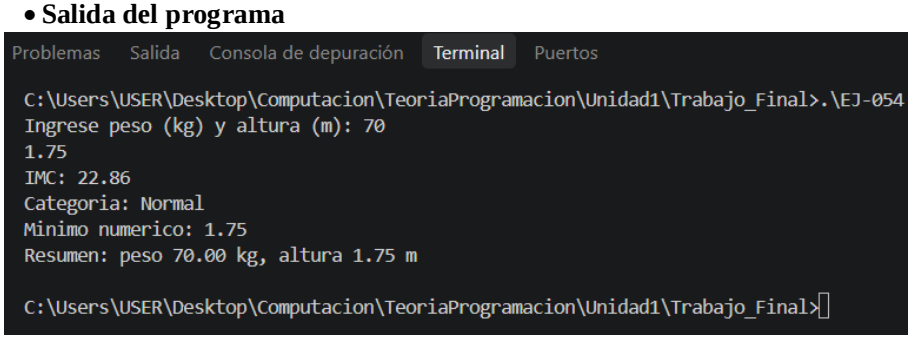


Figura 18. *Salida de la terminal del EJ-054*

Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-054

- **Conclusiones del algoritmo**
  - Comprendí como utilizar varias condiciones para clasificar un resultado en diferentes categorías según los límites.
  - Aprendí a como calcular la masa corporal mediante la formula que usa el peso y la altura.

- **Enunciado del algoritmo:**

EJ-055 (Avanzado)

En una jugueteria, diseñe un algoritmo que lea un numero entero de 3 cifras por teclado. Use constantes literales 10 y 100 con operadores division entera y modulo para extraer las centenas, decenas y unidades. Asignelas a variables c, d, u. Construya el numero invertido con la expresion aritmetica (u * 100 + d * 10 + c). Evalúe la expresion logica (numero == invertido) para determinar si es capicua. Muestre por pantalla el numero, el invertido y si es capicua. Ademas, calcule el valor minimo de todas las variables utilizadas. Asegurese de que las expresiones logicas esten bien parentizadas.

- **Análisis del Problema:**

Constantes literales: 10 y 100.

Datos de entrada:

Un número entero de tres cifras (numero).

Proceso:

c = numero / 100

d = (numero / 10) MOD 10

u = numero MOD 10

invertido = u * 100 + d * 10 + c

Comprobar si numero == invertido y encontrar el mínimo entre los valores numéricos.

Datos de salida:

Mensaje con el número original, invertido, si es capicúa y mínimo o aviso de número inválido.

Variables y tipos:

Como enteros numero, c, d, u, invertido y minimo

Como lógico es_capicua

Constantes literales enteras de 10 y 100

- **Diagrama de flujo:**

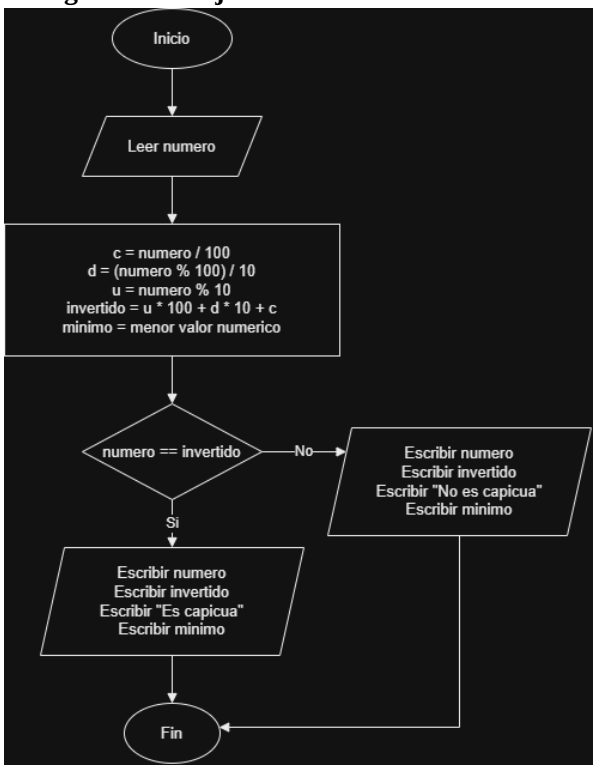


Figura 19. *Diagrama de flujo del EJ-055*

Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-055

- **Pseudocódigo:**

Inicio

Definir numero, c, d, u, invertido, minimo Como Entero

Definir es_capicua Como Logico

Escribir "Ingrese un numero de tres cifras:"

Leer numero

Si numero >= 100 Y numero <= 999 Entonces

c = numero DIV 100

```
d = (numero DIV 10) MOD 10
u = numero MOD 10
invertido = u * 100 + d * 10 + c
es_capicua = (numero == invertido)
minimo = numero
Si invertido < minimo Entonces minimo = invertido FinSi
Si c < minimo Entonces minimo = c FinSi
Si d < minimo Entonces minimo = d FinSi
Si u < minimo Entonces minimo = u FinSi
Escribir "Numero: ", numero
Escribir "Invertido: ", invertido (con tres posiciones)
Si es_capicua Entonces Escribir "Capicua: Si"
Sino Escribir "Capicua: No"
FinSi
Escribir "Minimo numerico: ", minimo
Sino
Escribir "El numero debe tener tres cifras positivas"
FinSi
Fin
```

- **Prueba de Escritorio:**

Casos	numero	c	d	u	invertido	minimo	Condición
1	121	1	2	1	121	1	Capicúa (121 = 121)
2	352	3	5	2	253	2	No capicúa (352 != 253)

Tabla 10. *Prueba de escritorio del EJ-055*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>
#include <stdbool.h>

void main() {
   int numero, c, d, u, invertido, minimo = 0;
   const int CIEN = 100;
   const int DIEZ = 10;
   bool es_capicua = false;
   //Entrada
   printf("Ingrese un numero entero de 3 cifras: ");
   scanf("%d", &numero);
   //Proceso y salida
   if (numero >= 100 && numero <= 999) {
      c = numero / CIEN;
      d = (numero / DIEZ) % DIEZ;
      u = numero % DIEZ;
      invertido = u * CIEN + d * DIEZ + c;
      es_capicua = (numero == invertido);
      minimo = numero;
      if (invertido < minimo) minimo = invertido;
      if (c < minimo) minimo = c;
      if (d < minimo) minimo = d;
      if (u < minimo) minimo = u;
      printf("Numero: %d\nInvertido: %03d\n", numero, invertido);
      printf("Capicua: %s\nMinimo numerico: %d\n",
            es_capicua ? "Si" : "No", minimo);
```

```
    } else {
        printf("El numero debe tener 3 cifras positivas\n");
    }
}
```

- **Salida del programa**

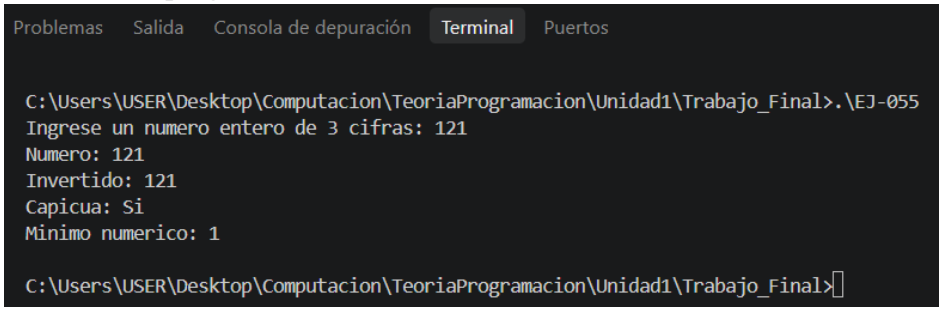


Figura 20. *Salida de la terminal del EJ-055*
Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-055

- **Conclusiones del algoritmo**
  - Utilicé una condición para comparar el numero normal y el invertido para determinar si era capicúa o no.
  - Comprendí como construir un numero invertido usando las cifras obtenidas en el orden contrario.
- **Enunciado del algoritmo:**

EJ-101 (Medio)
En una universidad, diseñe un algoritmo que lea dos numeros reales por teclado. Use la constante literal 2 como divisor. Calcule el promedio con la expresion aritmetica ((num1 + num2) / 2) y asignelo a promedio. Evalúe la expresion relacional (promedio > 0). Muestre por pantalla el promedio calculado. Identifique el tipo de dato de cada variable. Adicionalmente, calcule la potencia del resultado elevado a la 4. Incluya una seccion que explique brevemente el proposito del algoritmo.

- **Análisis del Problema:**

Constante literal: 2
Datos de entrada:
Dos números reales (numero1 y numero2)
Proceso:
promedio = (numero1 + numero2) / 2
Comprobar si promedio > 0
potencia = promedio * promedio * promedio * promedio
Datos de salida:
Mensaje con el promedio, si es positivo o no y el resultado de la potencia cuarta.
Variables y tipos:
Como reales numero1, numero2, promedio y potencia
Constante literal de 2
Propósito: calcular el promedio de dos números, comprobar su signo y elevarlo a la cuarta potencia

- **Diagrama de flujo:**

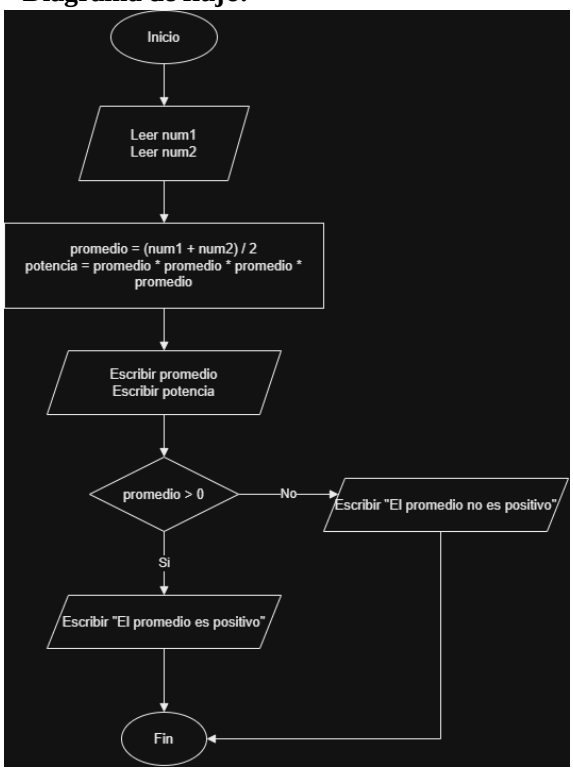


Figura 21. *Diagrama de flujo del EJ-101*

Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-101

- **Pseudocódigo:**

```
Inicio
Definir numero1, numero2, promedio, potencia Como Real
Escribir "Ingrese dos numeros reales:"
Leer numero1
Leer numero2
promedio = (numero1 + numero2) / 2
potencia = promedio * promedio * promedio * promedio
Escribir "Promedio: ", promedio
Si promedio > 0 Entonces
Escribir "El promedio es positivo"
Sino
Escribir "El promedio no es positivo"
FinSi
Escribir "Potencia cuarta: ", potencia
Fin
```

- **Prueba de Escritorio:**

Casos	num1	num2	promedio	potencia	Condición
1	8.2	3.8	6.00	1296.00	Positivo (6 > 0)
2	-4	2	-1.00	1.00	No positivo (-1 < 0)

Tabla 11. *Prueba de escritorio del EJ-101*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>

void main() {
   float numero1, numero2, promedio, potencia = 0.0;
   const int DIVISOR = 2;
   //Entrada
   printf("Ingrese dos numeros reales: ");
   scanf("%f %f", &numero1, &numero2);
```

```
    //Proceso
    promedio = (numero1 + numero2) / DIVISOR;
    potencia = promedio * promedio * promedio * promedio;
    //Salida
    printf("Promedio: %.2f\n", promedio);
    if (promedio > 0)
        printf("El promedio es positivo\n");
    else if (promedio < 0)
        printf("El promedio es negativo\n");
    else
        printf("El promedio es neutro\n");
    printf("Potencia cuarta: %.2f\n", potencia);
}
```

- **Salida del programa**

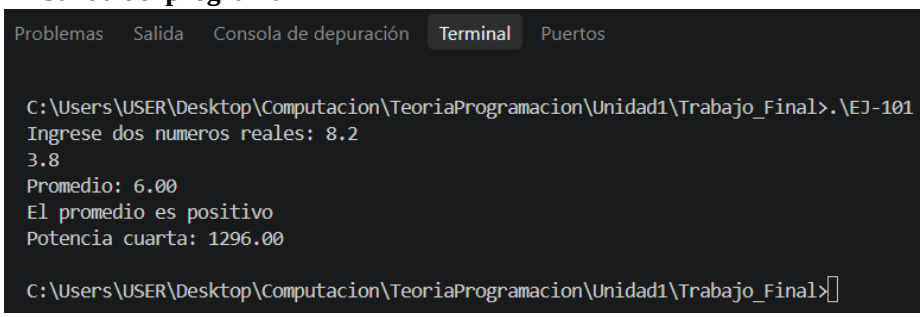


Figura 22. *Salida de la terminal del EJ-101*

Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-101

- **Conclusiones del algoritmo**
  - Comprendí como usar una potencia cuarta de un numero mediante multiplicaciones sucesivas, sin necesidad de funciones adicionales.
  - Utilicé una condición para comprobar si el promedio era negativo, positivo o neutro relacionando el resultado matemático con la expresión lógica.

- **Enunciado del algoritmo:**

EJ-102 (Medio)

En una tienda de electronica, cree un algoritmo que lea dos numeros enteros z y s por teclado. Evalúe la expresion logica (s != 0) para evitar division por cero. Calcule el cociente entero con (z / s) y el residuo con (z % s). Asigne los resultados a cociente y residuo. Muestre por pantalla ambos valores. Adicionalmente, calcule la diferencia entre el resultado y el promedio. Muestre al inicio una lista de las variables que se utilizaran.

- **Análisis del Problema:**

Datos de entrada:

Dos números enteros (z y s)

Proceso:

Comprobar que s != 0 para evitar la división entre cero.

cociente = z / s

residuo = z MOD s

promedio = (z + s) / 2.0

diferencia = cociente - promedio

Datos de salida:

Lista de variables, cociente, residuo, promedio y diferencia o aviso de división entre cero.

Variables y tipos:

Como enteros z, s, cociente y residuo

Como reales promedio y diferencia

Para el cálculo adicional, diferencia = cociente - promedio.

- **Diagrama de flujo:**

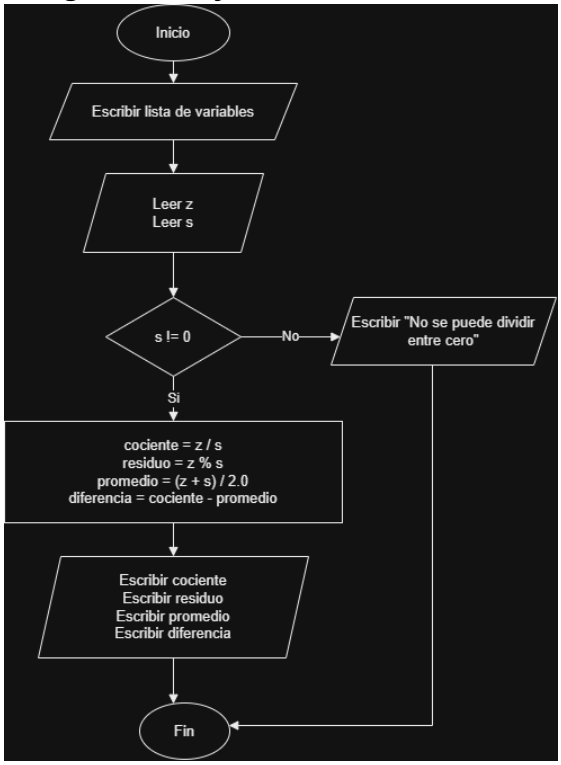


Figura 23. *Diagrama de flujo del EJ-102*

Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-102

- **Pseudocódigo:**

```
Inicio
Definir z, s, cociente, residuo Como Entero
Definir promedio, diferencia Como Real
Escribir "Ingrese z y s:"
Leer z
Leer s
Si s != 0 Entonces
cociente = z / s
residuo = z MOD s
promedio = (z + s) / 2.0
diferencia = cociente - promedio
Escribir "Cociente: ", cociente
Escribir "Residuo: ", residuo
Escribir "Promedio: ", promedio
Escribir "Diferencia: ", diferencia
Sino
Escribir "No se puede dividir entre cero"
FinSi
Fin
```

- **Prueba de Escritorio:**

Casos	z	s	cociente	residuo	promedio	diferencia	Condición
1	17	5	3	2	11.00	-8.00	s != 0: Sí
2	25	4	6	1	14.50	-8.50	s != 0: Sí

Tabla 12. *Prueba de escritorio del EJ-102*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>

void main() {
    int z, s, cociente, residuo = 0;
```

```
    float promedio, diferencia = 0.0;
    const float DIVISOR = 2.0;
    //Entrada
    printf("Ingrese dos numeros enteros: ");
    scanf("%d %d", &z, &s);
    //Proceso y salida
    if (s != 0) {
        cociente = z / s;
        residuo = z % s;
        promedio = (z + s) / DIVISOR;
        diferencia = cociente - promedio;
        printf("Cociente: %d\nResiduo: %d\nPromedio: %.2f\nDiferencia: %.2f\n", cociente, residuo, promedio,
diferencia);
    } else {
        printf("No se puede dividir entre cero\n");
    }
}
```

- **Salida del programa**

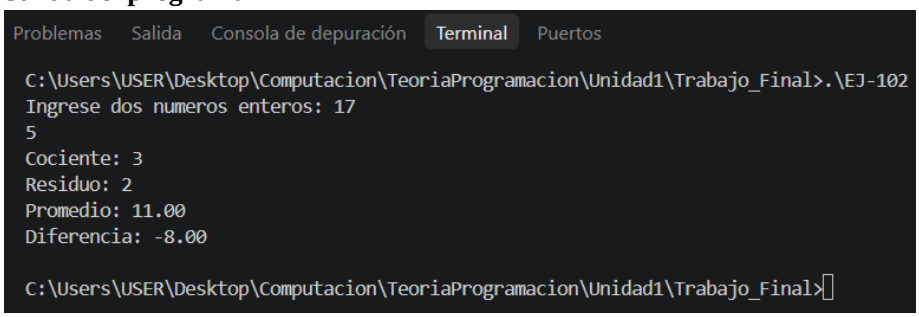


Figura 24. *Salida de la terminal del EJ-102*

Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-102

- **Conclusiones del algoritmo**
  - Aprendí a usar la división entera y el operador módulo para obtener el cociente y el residuo de dos números.
  - También aprendí a diferenciar el cociente entero del promedio real y a utilizar ambos resultados para calcular la diferencia solicitada.

- **Enunciado del algoritmo:**

EJ-103 (Medio)

En un gimnasio, construya un algoritmo que lea un numero entero a por teclado. Use constantes literales 2 y a-1 como rango. Evalúe las expresiones logicas (a > 1) y (no existe ningun divisor en el rango [2, a-1]). Asigne el resultado booleano a es_primo. Muestre por pantalla si el numero es primo o no. Identifique el tipo de dato de cada variable. Adicionalmente, calcule el porcentaje del resultado respecto a un valor de referencia. Justifique el tipo de dato simple elegido para cada variable.

- **Análisis del Problema:**

Datos de entrada:

Un número entero (a) y un valor real de referencia.

Proceso:

Comprobar si a > 1 y si tiene divisores entre 2 y a - 1

es_primo = verdadero si no existe ningún divisor exacto en ese rango.

porcentaje = a * 100 / referencia, cuando referencia > 0

Datos de salida:

Mensaje indicando si es primo o no y el porcentaje o aviso de referencia inválida.

Variables y tipos:

Como enteros a e i

Como lógico es_primo

Como reales referencia y porcentaje

Justificación: a e i se usan para valores enteros; es_primo es lógico, y referencia y porcentaje son reales porque pueden tener decimales.

- **Diagrama de flujo:**

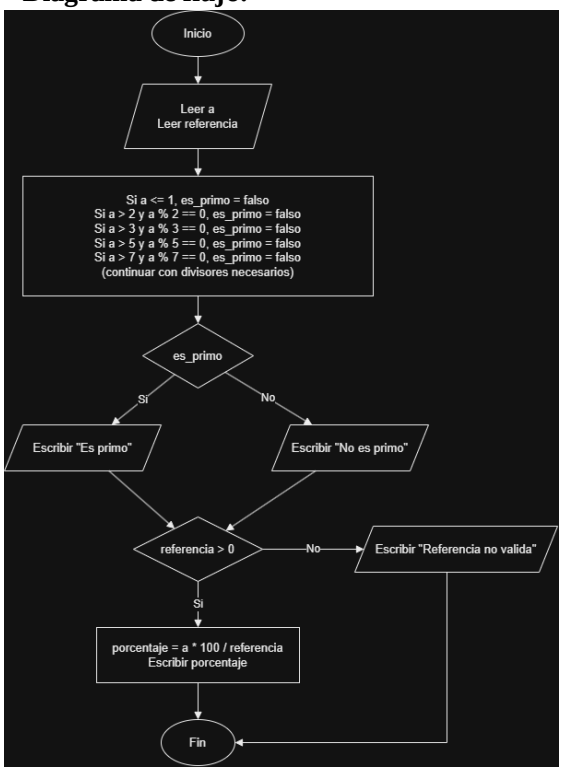


Figura 25. *Diagrama de flujo del EJ-103*

Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-103

- **Pseudocódigo:**

```
Inicio
Definir a, i Como Entero
Definir es_primo Como Logico
Definir referencia, porcentaje Como Real
Escribir "Ingrese un numero y la referencia:"
Leer a
Leer referencia
es_primo = Verdadero
Si a <= 1 Entonces
es_primo = Falso
Sino
Para i = 2 Hasta a - 1 Hacer
Si a MOD i == 0 Entonces
es_primo = Falso
FinSi
FinPara
FinSi
Si es_primo Entonces
Escribir "Es primo"
Sino
Escribir "No es primo"
FinSi
Si referencia > 0 Entonces
porcentaje = (a * 100) / referencia
Escribir "Porcentaje: ", porcentaje, "%"
Sino
Escribir "Referencia no valida"
FinSi
Fin
```

- **Prueba de Escritorio:**

Casos	a	referencia	es_primo	porcentaje	Condición
1	7	20	Sí	35.00%	a > 1; no tiene divisores
2	8	20	No	40.00%	a > 1; divisible entre 2

Tabla 13. *Prueba de escritorio del EJ-103*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>
#include <stdbool.h>

void main() {
   int num;
   bool es_primo = true;
   float ref, porct = 0;
   const int INICIO = 2;

   // Entrada
   printf("Ingrese un numero entero: ");
   scanf("%d", &num);
   const int FIN = num - 1;
   printf("Ingrese la referencia: ");
   scanf("%f", &ref);
   // Proceso
   if (num > 1000) {
      printf("Numero fuera del rango permitido\n");
   } else {
      if (num <= 1) es_primo = false;
      if (FIN >= INICIO && num % INICIO == 0) es_primo = false;
      if (FIN >= 3 && num % 3 == 0) es_primo = false;
      if (FIN >= 5 && num % 5 == 0) es_primo = false;
      if (FIN >= 7 && num % 7 == 0) es_primo = false;
      if (FIN >= 11 && num % 11 == 0) es_primo = false;
      if (FIN >= 13 && num % 13 == 0) es_primo = false;
      if (FIN >= 17 && num % 17 == 0) es_primo = false;
      if (FIN >= 19 && num % 19 == 0) es_primo = false;
      if (FIN >= 23 && num % 23 == 0) es_primo = false;
      if (FIN >= 29 && num % 29 == 0) es_primo = false;
      if (FIN >= 31 && num % 31 == 0) es_primo = false;
      // Salida
      if (es_primo)
         printf("Es primo\n");
      else
         printf("No es primo\n");
      if (ref > 0) {
         porct = num * 100.0 / ref;
         printf("Porcentaje: %.2f%%\n", porct);
      } else {
         printf("Referencia no valida\n");
      }
   }
}
```

- **Salida del programa**

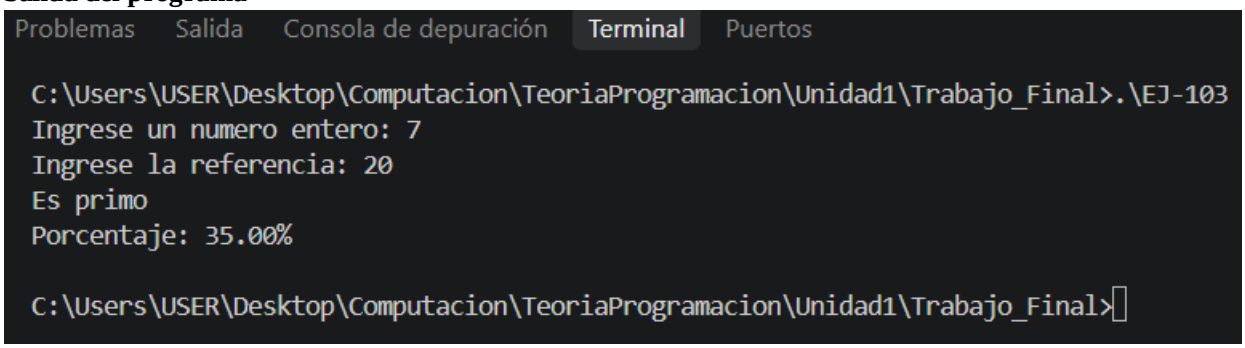


Figura 26. *Salida de la terminal del EJ-103*

Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-103

- **Conclusiones del algoritmo**
  - Una dificultad fue realizar las comprobaciones sin utilizar ciclos de repetición. Para resolverlo utilicé varias condiciones, aunque esta solución queda limitada al rango de números que permite comprobar el programa.
  - Aprendí que un número primo es aquel que es mayor que uno y solamente puede dividirse exactamente entre uno y él mismo.

- **Enunciado del algoritmo:**

EJ-104 (Medio)

En una floreria, diseñe un algoritmo que lea el capital (real), la tasa de interes anual (real) y el tiempo en años (entero). Use la constante literal 100. Calcule el interes con la expresion aritmetica (capital * tasa * tiempo / 100) y el monto total con (capital + interes). Asigne los resultados a interes_generado y monto_total. Muestre por pantalla el desglose. Adicionalmente, redondee el resultado a 2 decimales. Agregue una validacion para asegurar que los datos sean positivos.

- **Análisis del Problema:**

Constante literal: 100

Datos de entrada:

Capital y tasa de interés reales, y tiempo en años entero.

Proceso:

Comprobar que capital, tasa y tiempo sean positivos
interes_generado = capital * tasa * tiempo / 100
monto_total = capital + interes_generado
Mostrar los resultados con dos decimales

Datos de salida:

Mensaje con el capital, interés generado y monto total o aviso de datos inválidos.

Variables y tipos:

Como reales capital, tasa, interes_generado y monto_total
Como entero tiempo
Constante literal entera de 100

- **Diagrama de flujo:**

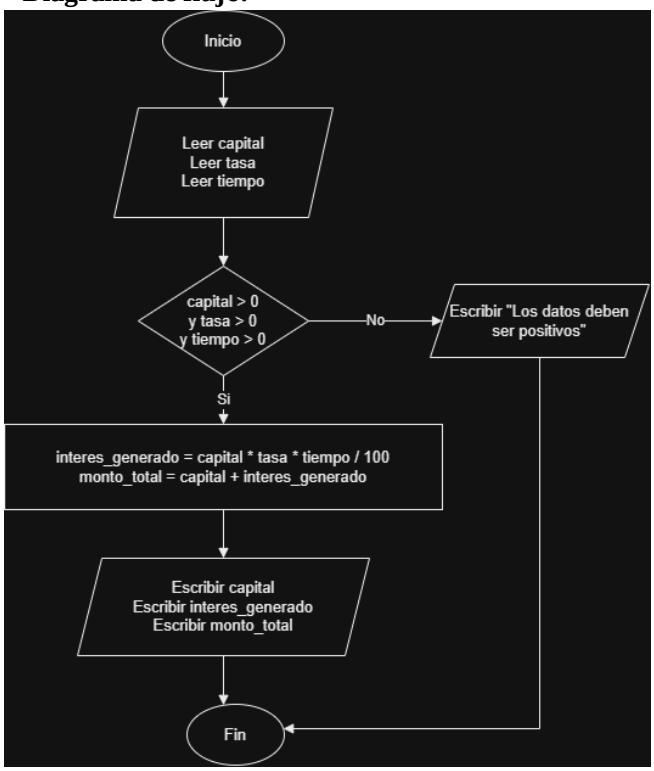


Figura 27. *Diagrama de flujo del EJ-104*

Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-104

- **Pseudocódigo:**

```
Inicio
Definir capital, tasa, interes_generado, monto_total Como Real
Definir tiempo Como Entero
Escribir "Ingrese capital, tasa anual y tiempo:"
Leer capital
Leer tasa
Leer tiempo
Si capital > 0 Y tasa > 0 Y tiempo > 0 Entonces
interes_generado = capital * tasa * tiempo / 100
monto_total = capital + interes_generado
Escribir "Capital: ", Redondear(capital, 2)
Escribir "Interes: ", Redondear(interes_generado, 2)
Escribir "Monto total: ", Redondear(monto_total, 2)
Sino
Escribir "Los datos deben ser positivos"
FinSi
Fin
```

- **Prueba de Escritorio:**

Casos	capital	tasa	tiempo	interes_ generado	monto_ total	Condición
1	1000	5	2	100.00	1100.00	Todos positivos
2	1234.56	3.5	3	129.6288 (129.63)	1364.1888 (1364.19)	Todos positivos

Tabla 14. *Prueba de escritorio del EJ-104*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>

void main() {
    const int PORCENTAJE = 100;
```

```
    float capital, tasa, interes_generado, monto_total = 0.0;
    int tiempo = 0;
    //Entrada
    printf("Ingrese capital, tasa anual y tiempo: ");
    scanf("%f %f %d", &capital, &tasa, &tiempo);
    //Proceso y salida
    if (capital > 0 && tasa > 0 && tiempo > 0) {
        interes_generado = capital * tasa * tiempo / PORCENTAJE;
        monto_total = capital + interes_generado;
        printf("Capital: %.2f\nInteres generado: %.2f\nMonto total: %.2f\n", capital, interes_generado, monto_total);
    } else {
        printf("Los datos deben ser positivos\n");
    }
}
```

- **Salida del programa**

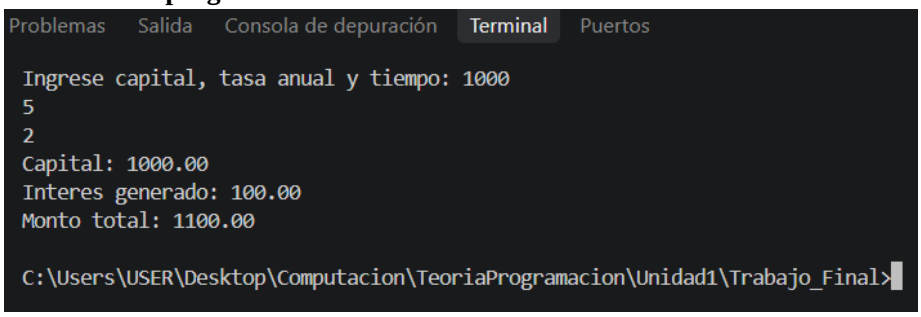


Figura 28. *Salida de la terminal del EJ-104*

Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-104

- **Conclusiones del algoritmo**
  - Aprendí a como calcular el interés generado por un capital utilizando una tasa anual y un tiempo determinado.
  - Comprendí cómo obtener el monto total sumando el capital inicial y el interés calculado, aplicando correctamente la fórmula matemática.

- **Enunciado del algoritmo:**

EJ-105 (Medio)

En un hotel, cree un algoritmo que lea tres calificaciones (reales) de un estudiante. Use la constante literal 2 como divisor. Identifique la menor con expresiones relacionales. Calcule el promedio de las dos mejores con la expresion aritmetica ((suma_total - menor) / 2). Asignelo a promedio. Muestre por pantalla el promedio final. Adicionalmente, calcule el resultado divido entre una constante simbolica. Verifique que las constantes esten declaradas al inicio del algoritmo.

- **Análisis del Problema:**

Constante simbólica: DIVISOR_EXTRA = 2.0

Datos de entrada:

Tres calificaciones reales (nota1, nota2 y nota3)

Proceso:

suma_total = nota1 + nota2 + nota3

Encontrar la menor de las tres notas

promedio = (suma_total - menor) / 2

resultado_dividido = promedio / DIVISOR_EXTRA

Datos de salida:

Mensaje con la menor nota, el promedio de las dos mejores y el resultado dividido

Variables y tipos:

Como reales nota1, nota2, nota3, suma_total, menor, promedio y resultado_dividido

Como constante real DIVISOR_EXTRA

- **Diagrama de flujo:**

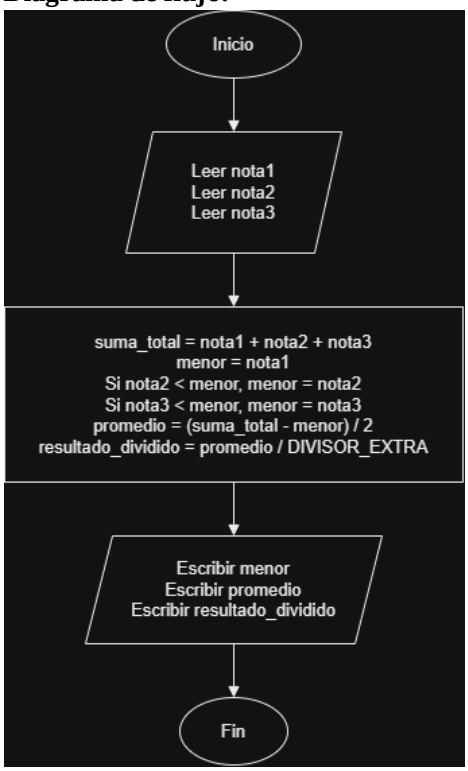


Figura 29. *Diagrama de flujo del EJ-105*
Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-105

- **Pseudocódigo:**

```
Inicio
Definir DIVISOR_EXTRA = 2.0 Como Constante Real
Definir nota1, nota2, nota3 Como Real
Definir suma_total, menor, promedio, resultado_dividido Como Real
Escribir "Ingrese tres calificaciones:"
Leer nota1
Leer nota2
Leer nota3
suma_total = nota1 + nota2 + nota3
menor = nota1
Si nota2 < menor Entonces
menor = nota2
FinSi
Si nota3 < menor Entonces
menor = nota3
FinSi
promedio = (suma_total - menor) / 2
resultado_dividido = promedio / DIVISOR_EXTRA
Escribir "Menor nota: ", menor
Escribir "Promedio final: ", promedio
Escribir "Resultado dividido: ", resultado_dividido
Fin
```

- **Prueba de Escritorio:**

Casos	nota1	nota2	nota3	suma_ total	menor	promedio	resultado_ dividido	Condición
1	8	9	7	24	7	8.50	4.25	9 < 8: No; 7 < 8: Sí
2	6	9	10	25	6	9.50	4.75	9 < 6: No; 10 < 6: No

Tabla 15. *Prueba de escritorio del EJ-105*
Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>

void main() {
   const float DIVISOR = 2.0;
   float nota1, nota2, nota3, suma_total, menor, promedio, resultado_dividido;
   //Entrada
   printf("Ingrese tres calificaciones: ");
   scanf("%f %f %f", &nota1, &nota2, &nota3);
   //Proceso
   suma_total = nota1 + nota2 + nota3;
   menor = nota1;
   if (nota2 < menor)
      menor = nota2;
   if (nota3 < menor)
      menor = nota3;
   promedio = (suma_total - menor) / 2;
   resultado_dividido = promedio / DIVISOR;
   //Salida
   printf("Menor nota: %.2f\nPromedio final: %.2f\nResultado dividido: %.2f\n", menor, promedio,
resultado_dividido);
}
```

- **Salida del programa**

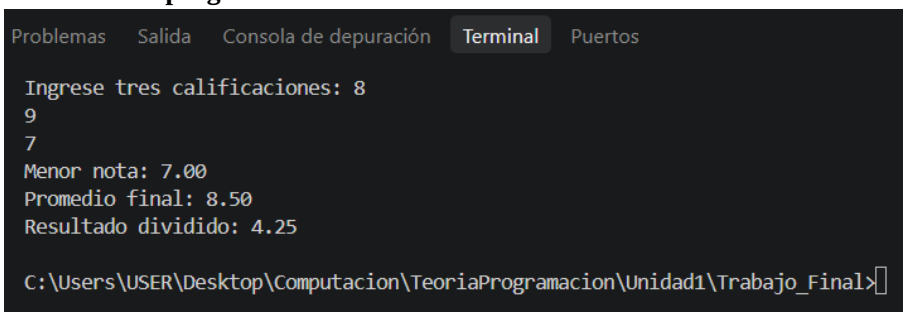


Figura 30. *Salida de la terminal del EJ-105*

Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-105

- **Conclusiones del algoritmo**
  - Aprendí a comparar tres calificaciones para identificar cuál de ellas tiene el menor valor.
  - Una de mis dificultades fue organizar las comparaciones para identificar correctamente la menor calificación.

- **Enunciado del algoritmo:**

EJ-201 (Medio)

En una tienda de ropa, construya un algoritmo que lea el largo, ancho y alto (tres reales) de un prisma rectangular. Calcule el volumen con la expresion aritmetica (largo * ancho * alto) y asignelo a volumen. Evalúe la expresion logica (largo > 0 AND ancho > 0 AND alto > 0). Muestre por pantalla el volumen del prisma. Ademas del calculo principal, multiplique el resultado por 1.5 como bono. Incluya una variable auxiliar para llevar un contador de operaciones.

- **Análisis del Problema:**

Constante literal: 1.5

Datos de entrada:

Tres medidas reales (largo, ancho y alto).

Proceso:

Comprobar que largo > 0, ancho > 0 y alto > 0
volumen = largo * ancho * alto
contador = contador + 1
bono = volumen * 1.5
contador = contador + 1

Datos de salida:

Mensaje con el volumen, bono y contador de operaciones o aviso de datos inválidos

Variables y tipos:

Como reales largo, ancho, alto, volumen y bono

Como entero contador

Constante literal real de 1.5

- **Diagrama de flujo:**

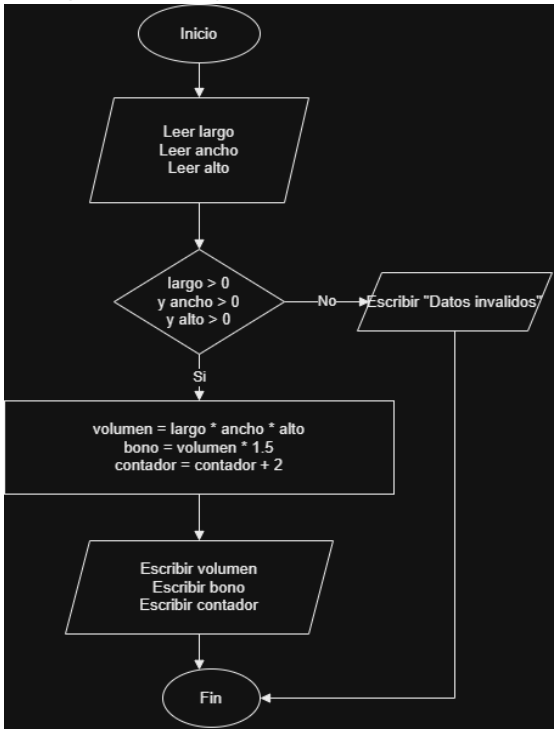


Figura 31. *Diagrama de flujo del EJ-201*

Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-201

- **Pseudocódigo:**

Inicio
Definir largo, ancho, alto, volumen, bono Como Real
Definir contador Como Entero
contador = 0
Escribir "Ingrese largo, ancho y alto:"
Leer largo
Leer ancho
Leer alto
Si largo > 0 Y ancho > 0 Y alto > 0 Entonces
volumen = largo * ancho * alto
contador = contador + 1
bono = volumen * 1.5
contador = contador + 1
Escribir "Volumen: ", volumen
Escribir "Bono: ", bono
Escribir "Operaciones: ", contador
Sino
Escribir "Las medidas deben ser positivas"
FinSi
Fin

- **Prueba de Escritorio:**

Casos	largo	ancho	alto	volumen	bono	contador	Condición
1	8	9	9	648.00	972.00	2	Todas las medidas > 0
2	2	3	4	24.00	36.00	2	Todas las medidas > 0

Tabla 16. *Prueba de escritorio del EJ-201*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>

void main() {
   float largo, ancho, alto, volumen, bono = 0.0;
   const float BONO = 1.5;
   int contador = 0;
   //Entrada
   printf("Ingrese largo, ancho y alto: ");
   scanf("%f %f %f", &largo, &ancho, &alto);
   //Proceso
   if (largo > 0 && ancho > 0 && alto > 0) {
      volumen = largo * ancho * alto;
      contador = contador + 1;
      bono = volumen * BONO;
      contador = contador + 1;
      printf("Volumen: %.2f\nBono: %.2f\nOperaciones: %d\n", volumen, bono, contador);
   } else {
      printf("Las medidas deben ser positivas\n");
   }
}
```

- **Salida del programa**

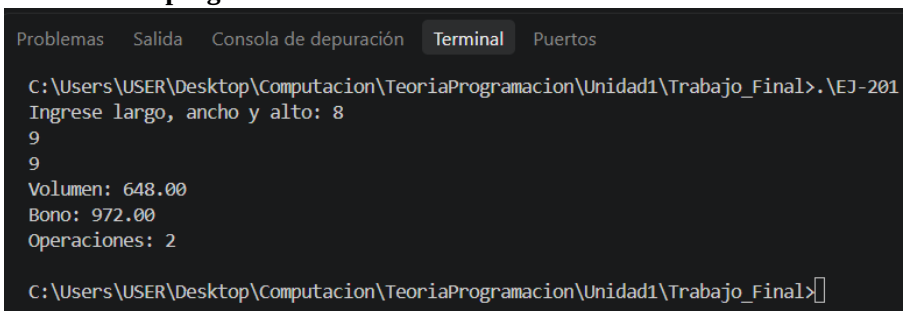


Figura 32. *Salida de la terminal del EJ-201*

Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-201

- **Conclusiones del algoritmo**
  - Aprendí a calcular el volumen de un prisma rectangular utilizando las tres medidas ingresadas.
  - Comprendí cómo utilizar una condición lógica para verificar que todas las medidas fueran positivas

- **Enunciado del algoritmo:**

EJ-202 (Medio)

En un taller mecanico, diseñe un algoritmo que lea tres numeros enteros z, s y val1 por teclado. Use la constante literal 3 como divisor. Calcule el promedio con la expresion aritmetica ((z + s + val1) / 3) y asignelo a promedio. Evalúe la expresion logica (promedio > 0). Muestre por pantalla el promedio. Ademas del calculo principal, calcule el resultado elevado al cubo. Asegurese de que las expresiones logicas esten bien parentizadas.

- **Análisis del Problema:**

Constante literal: 3.0

Datos de entrada:

Tres números enteros (z, s y val1)

Proceso:

promedio = (z + s + val1) / 3.0

cubo = promedio * promedio * promedio

Comparar el promedio con 0 para indicar si es positivo o no.

Datos de salida:

Mensaje con el promedio, su signo y el resultado elevado al cubo

Variables y tipos:

Como enteros z, s y val1

Como reales promedio y cubo

Constante literal de 3.0

- **Diagrama de flujo:**

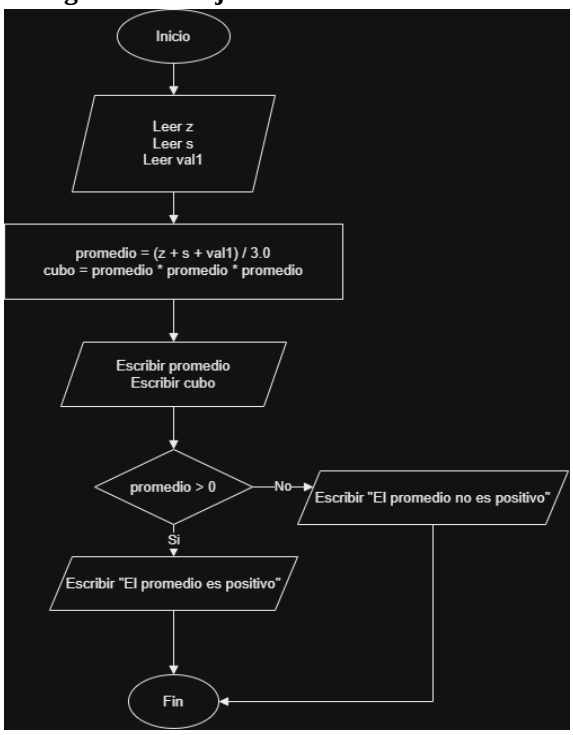


Figura 33. *Diagrama de flujo del EJ-202*

Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-202

- **Pseudocódigo:**

```
Inicio
Definir z, s, val1 Como Entero
Definir promedio, cubo Como Real
Escribir "Ingrese tres numeros enteros:"
Leer z
Leer s
Leer val1
promedio = (z + s + val1) / 3.0
cubo = promedio * promedio * promedio
Escribir "Promedio: ", promedio
Si (promedio > 0) Entonces
Escribir "El promedio es positivo"
Sino
Escribir "El promedio es negativo"
FinSi
Escribir "Promedio al cubo: ", cubo
Fin
```

- **Prueba de Escritorio:**

Casos	z	s	val1	promedio	cubo	Condición
1	6	7	6	6.33	254.04	Positivo (6.33 > 0)
2	-3	-6	0	-3.00	-27.00	Negativo (-3 < 0)

Tabla 17. *Prueba de escritorio del EJ-202*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>

void main() {
   int z, s, val1 = 0;
   float promedio, cubo = 0.0;
   const float DIVISOR = 3.0;
```

```
    //Entrada
    printf("Ingrese tres numeros enteros: ");
    scanf("%d %d %d", &z, &s, &val1);
    //Proceso
    promedio = (z + s + val1) / DIVISOR;
    cubo = promedio * promedio * promedio;
    printf("Promedio: %.2f\n", promedio);
    if (promedio > 0)
        printf("El promedio es positivo\n");
    else
        printf("El promedio es negativo\n");
    printf("Promedio al cubo: %.2f\n", cubo);
}
```

- **Salida del programa**

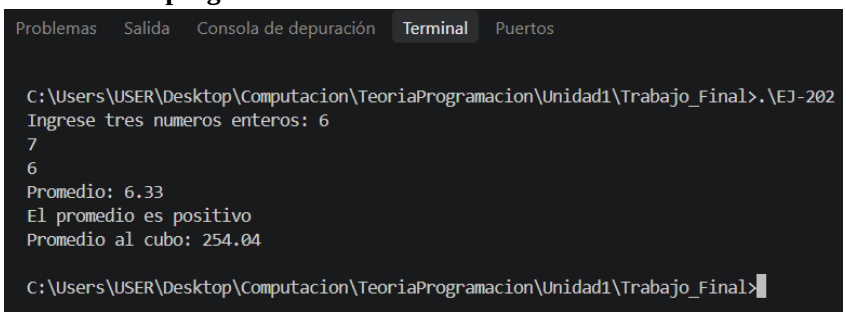


Figura 34. *Salida de la terminal del EJ-202*

Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-202

- **Conclusiones del algoritmo**
  - Comprendí cómo calcular el promedio al cubo mediante tres multiplicaciones del mismo resultado.
  - Una dificultad fue evitar que la división eliminara los decimales. Lo resolví utilizando 3.0 como divisor.

- **Enunciado del algoritmo:**

EJ-203 (Medio)

En una libreria, cree un algoritmo que lea un monto en soles (real). Use una constante simbolica TASA_CAMBIO = 0.27. Calcule el equivalente en dolares con la expresion aritmetica (soles * TASA_CAMBIO) y asignelo a dolares. Evalúe la expresion relacional (dolares > 0). Muestre por pantalla 'X soles equivalen a Y dolares'. Ademas del calculo principal, calcule el producto del resultado por PI. Asegurese de inicializar todas las variables antes de usarlas.

- **Análisis del Problema:**

Constantes simbólicas: TASA_CAMBIO = 0.27 y PI = 3.14159

Datos de entrada:

Un monto real expresado en soles.

Proceso:

dolares = soles * TASA_CAMBIO

producto = dolares * PI

Comprobar si dolares > 0

Datos de salida:

Mensaje con la equivalencia en dólares, el signo del resultado y el producto por PI.

Variables y tipos:

Como reales soles, dolares y producto

Como constantes reales TASA_CAMBIO y PI

- **Diagrama de flujo:**

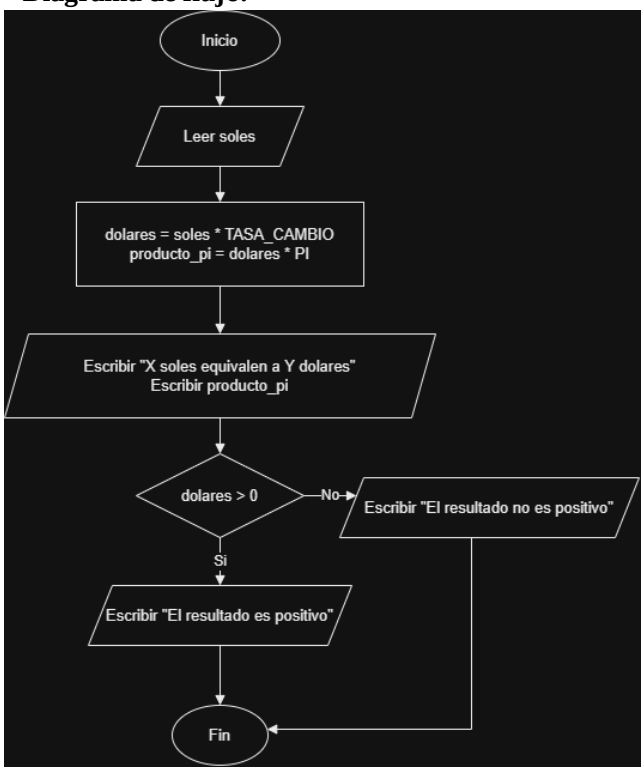


Figura 35. *Diagrama de flujo del EJ-203*
Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-203

- **Pseudocódigo:**

```
Inicio
Definir soles, dolares, producto Como Real
Definir TASA_CAMBIO = 0.27 Como Constante Real
Definir PI = 3.14159 Como Constante Real
Escribir "Ingrese el monto en soles:"
Leer soles
dolares = soles * TASA_CAMBIO
producto = dolares * PI
Escribir soles, " soles equivalen a ", dolares, " dolares"
Si dolares > 0 Entonces
Escribir "El resultado es positivo"
Sino
Escribir "El resultado es negativo"
FinSi
Escribir "Producto por PI: ", producto
Fin
```

- **Prueba de Escritorio:**

Casos	soles	dolares	producto	Condición
1	40	10.80	33.93	Positivo (10.80 > 0)
2	-10	-2.70	-8.48	Negativo (-2.70 < 0)

Tabla 18. *Prueba de escritorio del EJ-203*
Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>

void main() {
   const float TASA_CAMBIO = 0.27;
   const float PI = 3.14159;
   float soles, dolares, producto = 0.0;
   //Entrada
```

```
    printf("Ingrese el monto en soles: ");
    scanf("%f", &soles);
    dolares = soles * TASA_CAMBIO;
    producto = dolares * PI;
    //Salida
    printf("%.2f soles equivalen a %.2f dolares\n", soles, dolares);
    if (dolares > 0)
        printf("El resultado es positivo\n");
    else
        printf("El resultado es negativo\n");
    printf("Producto por PI: %.2f\n", producto);
}
```

- **Salida del programa**

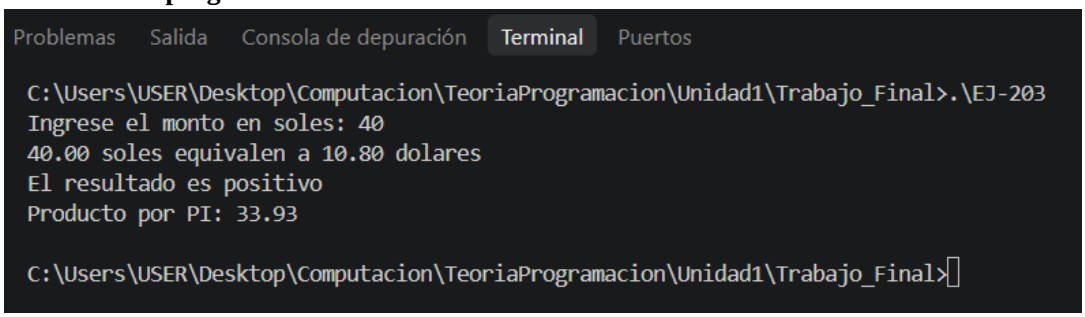


Figura 36. *Salida de la terminal del EJ-203*
Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-203

- **Conclusiones del algoritmo**
  - Aprendí a convertir una cantidad de soles a dólares utilizando una tasa de cambio en una constante.
  - Utilicé una condición para determinar si la cantidad convertida era positiva.

- **Enunciado del algoritmo:**

EJ-204 (Medio)
En una empresa de transporte, construya un algoritmo que lea el radio (real) y el angulo (real, en grados) de un sector circular. Use constantes simbolicas PI = 3.14159 y FACTOR = 360. Calcule el area con la expresion aritmetica ((PI * radio * radio * angulo) / FACTOR) y asignela a area_sector. Evalúe la expresion logica (radio > 0 AND angulo > 0). Muestre por pantalla el area del sector. Ademas, calcule el cuadrado del resultado principal. Agregue al final un resumen de los datos ingresados.

- **Análisis del Problema:**

Constantes simbólicas: PI = 3.14159 y FACTOR = 360
Datos de entrada:
Dos números reales (radio y angulo)
Proceso:
Comprobar que radio > 0 y angulo > 0
area_sector = (PI * radio * radio * angulo) / FACTOR
cuadrado = area_sector * area_sector
Datos de salida:
Mensaje con el área del sector, su cuadrado y resumen de los datos ingresados o aviso de datos inválidos.
Variables y tipos:
Como reales radio, angulo, area_sector y cuadrado
Como constantes reales PI y FACTOR

- **Diagrama de flujo:**

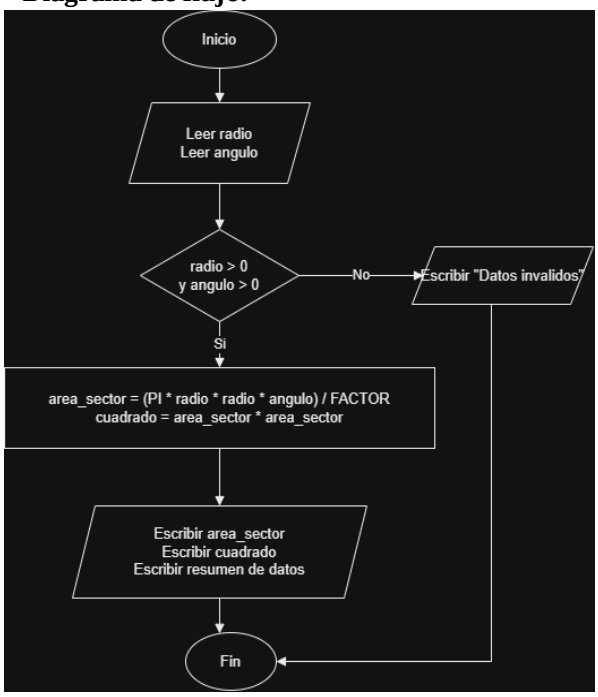


Figura 37. *Diagrama de flujo del EJ-204*

Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-204

- **Pseudocódigo:**

Inicio
Definir radio, angulo, area_sector, cuadrado Como Real
Definir PI = 3.14159 Como Constante Real
Definir FACTOR = 360 Como Constante Real
Escribir "Ingrese radio y angulo en grados:"
Leer radio
Leer angulo
Si radio > 0 Y angulo > 0 Entonces
area_sector = (PI * radio * radio * angulo) / FACTOR
cuadrado = area_sector * area_sector
Escribir "Area del sector: ", area_sector
Escribir "Cuadrado del area: ", cuadrado
Sino
Escribir "Radio y angulo deben ser positivos"
FinSi
Escribir "Datos: radio ", radio, ", angulo ", angulo
Fin

- **Prueba de Escritorio:**

Casos	radio	angulo	area_sector	cuadrado	Condición
1	2	90	3.14	9.87	Radio y ángulo > 0
2	3	180	14.14	199.86	Radio y ángulo > 0

Tabla 19. *Prueba de escritorio del EJ-204*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>

void main() {
   const float PI = 3.14159;
   const float FACTOR = 360;
   float radio, angulo, area_sector, cuadrado = 0.0;
   //Entrada
   printf("Ingrese radio y angulo en grados: ");
```

```
    scanf("%f %f", &radio, &angulo);
    //salida
    if (radio > 0 && angulo > 0) {
        area_sector = (PI * radio * radio * angulo) / FACTOR;
        cuadrado = area_sector * area_sector;
        printf("Area del sector: %.2f\nCuadrado del area: %.2f\n", area_sector, cuadrado);
        printf("Datos: radio %.2f, angulo %.2f grados\n", radio, angulo);
    } else {
        printf("Radio y angulo deben ser positivos\n");
        printf("Datos: radio %.2f, angulo %.2f grados\n", radio, angulo);
    }
}
```

- **Salida del programa**

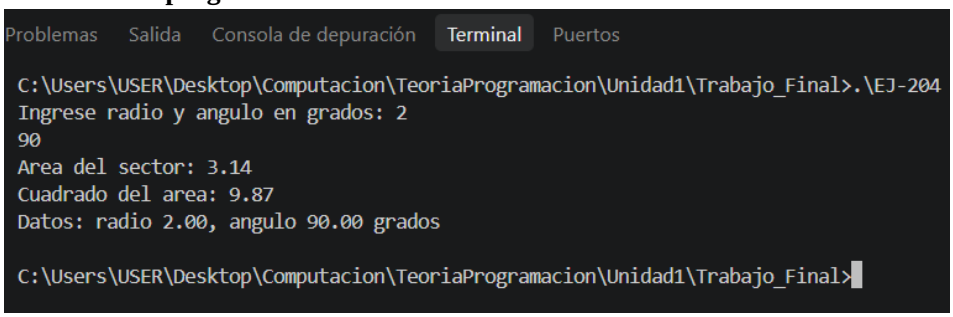


Figura 38. *Salida de la terminal del EJ-204*

Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-204

- **Conclusiones del algoritmo**
  - Utilicé condiciones para comprobar que el radio y el ángulo fueran positivos, evitando realizar cálculos que no cumplen los requisitos.
  - Aprendí a calcular el área de un sector circular utilizando el radio, el ángulo y las constantes necesarias.

- **Enunciado del algoritmo:**

EJ-205 (Medio)

En una zapateria, diseñe un algoritmo que lea un numero entero n por teclado. Use la constante literal 2. Evalúe las expresiones logicas (n > 0) y (n % 2 == 0). Asigne el resultado de combinarlas con AND a una variable es_par_positivo. Muestre por pantalla si el numero es par y positivo. Ademas, calcule el valor absoluto de la diferencia entre dos resultados. Identifique explicitamente los datos de entrada y la informacion de salida.

- **Análisis del Problema:**

Constante literal: 2

Datos de entrada:

Un número entero (n)

Proceso:

positivo = (n > 0)

par = (n MOD 2 == 0)

es_par_positivo = positivo Y par

diferencia = ABS(positivo - par)

Datos de salida:

Mensaje indicando si el número es par y positivo y valor absoluto de la diferencia.}

Variables y tipos:

Como entero n

Como enteros positivo, par, es_par_positivo y diferencia

Constante literal entera de 2

- **Diagrama de flujo:**

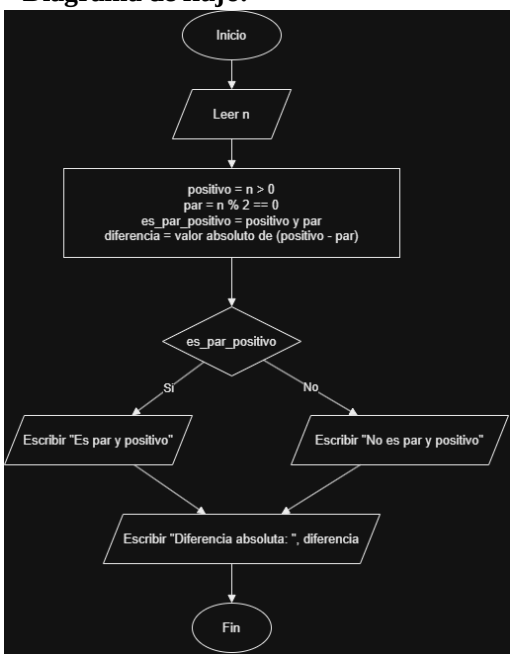


Figura 39. *Diagrama de flujo del EJ-205*

Nota: Captura de pantalla del diagrama de flujo hecho en draw.io del EJ-205

- **Pseudocódigo:**

Inicio
Definir n, diferencia Como Entero
Definir positivo, par, es_par_positivo Como Logico
Escribir "Ingrese un numero entero:"
Leer n
positivo = (n > 0)
par = (n MOD 2 == 0)
es_par_positivo = positivo Y par
diferencia = ABS(positivo - par)
Si es_par_positivo Entonces
Escribir "El numero es par y positivo"
Sino
Escribir "El numero no es par y positivo"
FinSi
Escribir "Diferencia absoluta: ", diferencia
Fin

- **Prueba de Escritorio:**

Casos	n	positivo	par	es_par_positivo	diferencia	Condición
1	8	1	1	1	0	Par y positivo
2	-8	0	1	0	1	No es par y positivo

Tabla 20. *Prueba de escritorio del EJ-205*

Nota: Tabla de prueba con los datos para la comprobación y sus resultados.

- **Codificación en lenguaje C**

```
#include <stdio.h>
#include <stdlib.h>

void main() {
   const int DIVISOR = 2;
   int n, positivo, par, es_par_positivo, diferencia = 0;
   //Entrada
   printf("Ingrese un numero entero: ");
   scanf("%d", &n);
   //Proceso
```

```
    positivo = (n > 0);
    par = (n % DIVISOR == 0);
    es_par_positivo = positivo && par;
    diferencia = abs(positivo - par);
    //Salida
    if (es_par_positivo)
        printf("El numero es par y positivo\n");
    else
        printf("El numero no es par y positivo\n");
    printf("Diferencia absoluta: %d\n", diferencia);
}
```

- **Salida del programa**

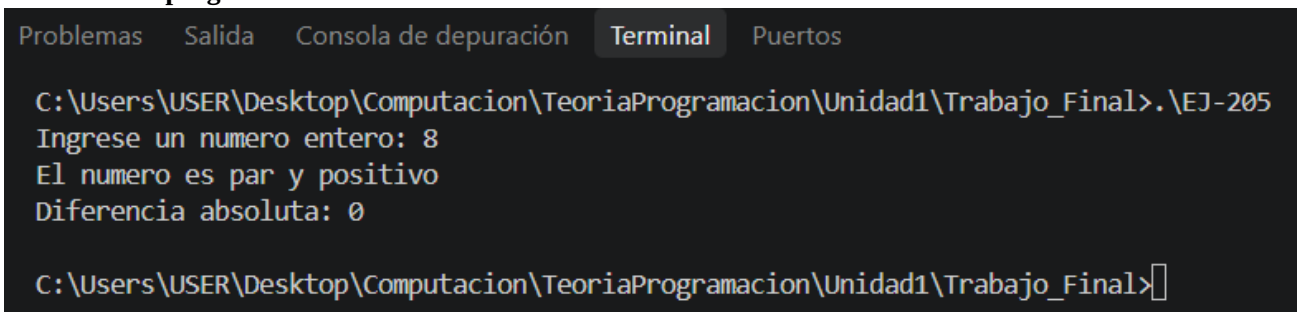


Figura 40. *Salida de la terminal del EJ-205*

Nota: Captura de pantalla de la salida de la terminal en VS code del EJ-205

- **Conclusiones del algoritmo**
  - Aprendí a combinar dos condiciones lógicas para determinar si un número es positivo y par al mismo tiempo.
  - También aprendí a utilizar la función abs() para obtener el valor absoluto de una diferencia.

5. Rúbrica de evaluación (sobre 10 puntos)

Se aplica a cada algoritmo desarrollado.

Criterio	Excelente	Bueno	Regular	Insuficiente	Punt.
Análisis del problema (0,5 pts)	**(0,5)** Identifica correctamente entradas, proceso, salidas y variables con su tipo de dato.	**(0,4)** Identifica entradas, proceso y salidas con errores mínimos.	**(0,2)** Identifica parcialmente los elementos del análisis.	**(0)** No presenta el análisis o este es incorrecto.	
Diagrama de flujo (2,0 pts)	**(2,0)** Simbología correcta y flujo lógico completo; el algoritmo resuelve el problema.	**(1,5)** Simbología mayormente correcta; lógica con pequeños errores que no afectan el resultado.	**(1,0)** Errores de simbología y/o lógica que afectan parcialmente la solución.	**(0–0,5)** Diagrama incompleto, con lógica incorrecta o no presenta.	
PUNTAJE TOTAL POR ALGORITMO					/ 10

Criterio	Excelente	Bueno	Regular	Insuficiente	Punt.
Pseudocódigo (1,5 pts)	**(1,5)** Estructura completa y sintaxis adecuada; corresponde al diagrama de flujo.	**(1,2)** Estructura completa con errores menores de sintaxis.	**(0,8)** Estructura incompleta o con errores de sintaxis frecuentes.	**(0–0,4)** Pseudocódigo incorrecto o no presenta.	
Prueba de escritorio (2,0 pts)	**(2,0)** Tabla completa, ordenada y con resultados correctos; usa mínimo dos casos de prueba.	**(1,5)** Tabla completa con algún error de cálculo menor.	**(1,0)** Tabla incompleta o con varios errores que no validan el algoritmo.	**(0–0,5)** No presenta la prueba de escritorio o está totalmente errada.	
Codificación en C (2,0 pts)	**(2,0)** Código funcional, compilado, comentado, con buenas prácticas e indentación.	**(1,5)** Código funcional con errores menores de estilo o comentarios.	**(1,0)** Código con errores que impiden compilar o lógica incorrecta.	**(0–0,5)** Código no funcional o no presenta.	
Salida del programa (1,0 pto)	**(1,0)** Incluye captura de la ejecución; los resultados coinciden con la prueba de escritorio.	**(0,8)** Incluye captura con diferencias mínimas respecto a la prueba de escritorio.	**(0,5)** Incluye captura pero no coincide con la prueba de escritorio.	**(0–0,3)** No incluye evidencia de la ejecución.	
Conclusiones (0,5 pts)	**(0,5)** Conclusiones bien redactadas, claras y coherentes; de elaboración propia y reflejan aprendizaje y reflexión sobre el proceso.	**(0,4)** Conclusiones adecuadas con errores menores de redacción u ortografía.	**(0,2)** Conclusiones superficiales, muy breves o con deficiencias de redacción.	**(0)** No presenta conclusiones, son copiadas o generadas con IA.	
Presentación y redacción (0,5 pts)	**(0,5)** Documento ordenado, con portada, formato claro y redacción adecuada.	**(0,4)** Documento ordenado con errores menores de formato.	**(0,2)** Documento desordenado o con deficiencias de formato.	**(0)** Documento sin estructura ni presentación adecuada.	
PUNTAJE TOTAL POR ALGORITMO					**/ 10**

Escala de calificación

9,0 – 10	Domina el desarrollo de algoritmos
8,0 – 8,9	Alcanza los aprendizajes esperados

7,0 – 7,9	Está próximo a alcanzar los aprendizajes
Menos de 7,0	No alcanza los aprendizajes esperados

6. Observaciones adicionales

- Si se desarrollan **N algoritmos**, la nota final será el **promedio** de los puntajes obtenidos en cada uno.
- La **coherencia** entre el diagrama de flujo, el pseudocódigo, la prueba de escritorio, el código C y las conclusiones será un aspecto clave de la evaluación.
- El uso de **inteligencia artificial** o la copia de trabajos ajenos será calificado con **0** y notificado según el reglamento institucional.

Declaración de autoría

Declaro que el presente trabajo es de mi **autoría**, que **no se utilizó inteligencia artificial** (ni otra herramienta generadora de contenido) ni copia de terceros para su elaboración, y que conozco y acepto las sanciones establecidas en el reglamento institucional en caso de incumplimiento.



Firma del estudiante **Docente**

Repositorio de GitHub:

Teoria-Programacion/Unidad1/Trabajo_Final at main · iker-17/Teoria-Programacion